http://pubsonline.informs.org/journal/ijoc

INFORMS JOURNAL ON COMPUTING
Vol. 33, No. 1, Winter 2021, pp. 51–71
ISSN 1091-9856 (print), ISSN 1526-5528 (online)

Stochastic Decomposition for Two-Stage Stochastic Linear Programs with Random Cost Coefficients

Harsha Gangammanavar,[a] Yifan Liu,[b] Suvrajeet Sen[c]

[a] Southern Methodist University, Dallas, Texas 75275; [b] 84.51°, Cincinnati, Ohio 45202; [c] University of Southern California, Los Angeles, California 90089

Contact: harsha@smu.edu, https://orcid.org/0000-0003-4389-5433 (HG); yifanl@usc.edu (YL); s.sen@usc.edu, https://orcid.org/0000-0002-6285-8833 (SS)

Received: October 5, 2018
Revised: March 12, 2019; June 21, 2019
Accepted: July 21, 2019
Published Online in Articles in Advance: May 28, 2020

https://doi.org/10.1287/ijoc.2019.0929

Copyright: © 2020 INFORMS

Abstract. Stochastic decomposition (SD) has been a computationally effective approach to solve large-scale stochastic programming (SP) problems arising in practical applications. By using incremental sampling, this approach is designed to discover an appropriate sample size for a given SP instance, thus precluding the need for either scenario reduction or arbitrary sample sizes to create sample average approximations (SAA). When compared with the solutions obtained using the SAA procedure, SD provides solutions of similar quality in far less computational time using ordinarily available computational resources. However, previous versions of SD were not applicable to problems with randomness in second-stage cost coefficients. In this paper, we extend its capabilities by relaxing this assumption on cost coefficients in the second stage. In addition to the algorithmic enhancements necessary to achieve this, we also present the details of implementing these extensions, which preserve the computational edge of SD. Finally, we illustrate the computational results obtained from the latest implementation of SD on a variety of test instances generated for problems from the literature. We compare these results with those obtained from the regularized L-shaped method applied to the SAA function of these problems with different sample sizes.

History: Accepted by Andrea Lodi, Area Editor for Design & Analysis of Algorithms—Discrete.
Funding: This work was supported by the Air Force Office of Scientific Research [FA9550-15-1-0267]; the Division of Civil, Mechanical and Manufacturing Innovation [1538605, 1822327]; and the Division of Electrical, Communications and Cyber Systems [1548847].

Keywords: stochastic programming • stochastic decomposition • sample average approximation • two-stage models with random cost coefficients • sequential sampling

1. Introduction

The two-stage stochastic linear programming problem (2-SLP) can be stated as

$$\begin{aligned} \min \quad & f(\boldsymbol{x}) := \boldsymbol{c}^\top \boldsymbol{x} + \mathbb{E}[h(\boldsymbol{x}, \tilde{\boldsymbol{\omega}})] \\ \text{s.t.} \quad & \boldsymbol{x} \in \mathcal{X} := \{\boldsymbol{x} | \boldsymbol{A}\boldsymbol{x} \le \boldsymbol{b}\} \subseteq \mathcal{R}^{n_1}, \end{aligned} \tag{1a}$$

where the recourse function is defined as follows:

$$\begin{aligned} h(\boldsymbol{x}, \boldsymbol{\omega}) := \min \quad & \boldsymbol{d}(\boldsymbol{\omega})^\top \boldsymbol{y} \\ \text{s.t.} \quad & \boldsymbol{D}(\boldsymbol{\omega})\boldsymbol{y} = \boldsymbol{\xi}(\boldsymbol{\omega}) - \boldsymbol{C}(\boldsymbol{\omega})\boldsymbol{x} \\ & \boldsymbol{y} \ge 0, \boldsymbol{y} \in \mathcal{R}^{n_2}. \end{aligned} \tag{1b}$$

The random variable $\tilde{\boldsymbol{\omega}}$ is defined on a probability space $(\Omega, \mathcal{F}, \mathcal{P})$, where the sample space $\Omega \subseteq \mathcal{R}^{r_2}$, and $\boldsymbol{\omega}$ denotes a realization of this random variable. The problem statement allows one or more elements of data $(\boldsymbol{d}, \boldsymbol{D}, \boldsymbol{\xi}, \boldsymbol{C})$ to depend on the random variable. Moreover, the statement accommodates both continuous as well as discrete random variables. Computational approaches to address this formulation, however, have largely focused on problems with finite support, for which uncertainty is represented using a set of realizations $\{\boldsymbol{\omega}^1, \boldsymbol{\omega}^2, \ldots, \boldsymbol{\omega}^N\}$ with their respective probabilities $p^j, j = 1, \ldots, N$ that are assumed to be known. When N is small, a deterministic equivalent problem can be formulated and solved using off-the-shelf LP solvers.

In problems with continuous random variables, calculation of the expectation functional involves a multidimensional integral in high dimension. For such problems as well as those in which the number of possible realizations is astronomically large, sampling-based methods offer a computationally viable means for obtaining a reasonable approximation. When sampling precedes the optimization step (i.e., external sampling), the objective function in (1a) is replaced by its so-called sample average approximation (SAA) using a fixed number of outcomes. As an alternative to choosing the sample size, one can allow the algorithm to dynamically determine a sufficiently large sample size during optimization. We refer to such a coupling of sampling and optimization as sequential or internal sampling, and this forms the basis for the stochastic decomposition (SD) algorithm (Higle and Sen 1991, 1994).

The SD algorithm allows for seamlessly incorporating new outcomes to improve the approximation of the expected recourse function without having to restart the optimization. In order to accomplish this, the algorithm continuously gathers relevant information during the course of its run and efficiently reutilizes this information in future iterations. These features have enabled SD to provide high-quality solutions in far less computational time using ordinary desktop and laptop computers when compared with the methods that involve using the SAA function to obtain solutions of similar quality. This was demonstrated on a suite of stochastic programming (SP) instances in Sen and Liu (2016) and on a real-scale power system operations planning problem in Gangammanavar et al. (2016).

Nevertheless, previous research on SD, including the most recent work of Sen and Liu (2016), focuses on 2-SLPs in which uncertainty affects only the right-hand side elements of Subproblem (1b), that is, elements of vector $\boldsymbol{\xi}$ and matrix $\boldsymbol{C}$. In this paper, we extend the capability of SD to address 2-SLPs with random cost coefficients, making it applicable to problems in which parameters $(\boldsymbol{d}, \boldsymbol{\xi}, \boldsymbol{C})$ are governed by the random variable $\tilde{\boldsymbol{\omega}}$.

In classical approaches to solve 2-SLPs—including the L-shaped method (Van Slyke and Wets 1969)—the subproblems for all realizations are solved in every iteration, and the outer linearization generated in an iteration is based on dual multipliers computed only in that iteration. Hence, when compared with the computational workload for 2-SLPs with deterministic second-stage cost coefficients, an instance with random cost coefficients does not pose any *additional* difficulty for such methods. On the other hand, the reliance on information discovered in earlier iterations to generate SD approximations and, in particular, the repeated use of previously discovered dual vertices can no longer be used without modification when cost coefficients have random elements. In light of this, the main contributions of this work are as follows:

- We show that, in the presence of random cost coefficients, dual vertices can be represented by a decomposition consisting of a deterministic representation together with a shift vector that depends on the realization of the random variable. This shift vector can be applied when the elements of $\boldsymbol{d}(\boldsymbol{\omega})$ are both dependent or independent. Moreover, the dimension of the shift vector depends on the number of random elements in $\boldsymbol{d}$. For instances in which the number of random elements is small, the dimension of shift vectors is also small, thus allowing SD to retain many of the advantages available in the case of fixed second-stage cost coefficients.
- We design extensions of previous SD computational data structures to accommodate the introduction of random cost coefficients without sacrificing most of the computational advantage of SD. However, the number of random cost coefficients does have a negative impact on the algorithm's performance. In view of this, we also identify optimization model structures that help us to overcome these negative impacts.
- We design new instances with random cost coefficients that can be used to benchmark SP algorithms. We also present computational results obtained from the enhanced SD algorithm and compare them to those obtained using the regularized L-shaped method applied to the SAA function of varying sizes.

The rest of the paper is organized as follows. In Section 2, we present a succinct review of the SD algorithm focusing on the principal steps involved in computing the approximations of the first-stage objective function. In Section 3, we formalize all of the key algorithmic ideas necessary to enable the SD algorithm to address problems with random cost coefficients. Our discussion also includes notes on challenges associated with implementing these ideas and our approach to address them. This is followed by a narration of our computational experience in Section 5 on both small- and large-scale instances.

2. Background: Stochastic Decomposition

Before we present our work, a brief overview of the SD algorithm is in order. We refer the reader to Higle and Sen (1991, 1994) and Sen and Liu (2016) for detailed mathematical expositions of this algorithm. Our discussion here focuses on the construction of lower bounding approximations of the expected recourse function in (1a) with an eye toward computer implementation. We present a high-level description of the regularized SD algorithm while ignoring the algorithmic details regarding incumbent update (used in the proximal term of the regularized version) and stopping rules. In particular, we focus on the information that is collected during the course of the algorithm and discuss how this information is utilized to generate/update the approximations in a computationally efficient manner. In the remainder of this paper, the computer programming constructs are presented in `typewriter` font to distinguish them from the mathematical constructs.

We first state the assumptions for the 2-SLP models as required by previous versions of SD.

Assumption 1. *The set of first-stage solutions $\mathcal{X}$ and the set of outcomes Ω are compact.*

Assumption 2. *The 2-SLP model satisfies the fixed recourse property; in other words, the recourse matrix $\boldsymbol{D}$ is deterministic.*

Assumption 3. *There exists an $L > -\infty$ such that $L \leq h(\boldsymbol{x}, \tilde{\boldsymbol{\omega}}) < \infty$ almost surely.*

Assumption 4. *The cost coefficient vector d of Subproblem (1b) is deterministic.*

Note that $h(x, \tilde{\omega}) < \infty$ under Assumption 3 implies that our 2-SLP model conforms with the relatively complete recourse assumption of SP. Further, a finite lower bound on the recourse value implies that the dual feasible set is nonempty almost surely. We denote outcomes of random variables using a superscript over the parameter's name. For instance ξ^j and C^j are equivalent to $\xi(\omega^j)$ and $C(\omega^j)$, respectively. The letter k is reserved to denote the iteration counter of the algorithm.

2.1. Algorithmic Description

The principal steps of regularized SD are presented in Algorithm 1. We begin our description at iteration k when we have access to a candidate solution x^k, an incumbent solution $\hat{x}^k$, and the current approximation of first-stage objective function $f^k(x)$. The iteration begins by sampling an outcome ω^k of the random variable $\tilde{\omega}$ independently of all prior samples $\{\omega^j\}_{j=1}^{k-1}$. If this outcome is encountered for the first time, it is added to the collection of unique outcomes $\mathbb{O}^{k-1} := \{\omega^1, \omega^2, \ldots\}$ observed during the first $(k-1)$ iterations of the algorithm. The current candidate solution x^k and outcome ω^k are used to evaluate the recourse function $h(x^k, \omega^k)$ by solving the subproblem (1b) to optimality. The resulting optimal dual solution π_k^k is added to the collection of previously discovered dual vertices $\mathcal{V}_0^{k-1}$, that is, $\mathcal{V}_0^k = \mathcal{V}_0^{k-1} \cup \{\pi_k^k\}$. Using the outcomes in $\mathbb{O}^k$, one may define an SAA function

$$F^k(x) = c^\top x + \frac{1}{k}\sum_{j=1}^{k} h(x, \omega^j), \tag{2}$$

which uses a sample size of k. In iteration k, the SD algorithm creates an approximation $f^k(x)$, which satisfies $f^k(x) \le F^k(x)$. In order to see how this is accomplished, we make the following observations:

1. Under Assumptions 2–4, the dual feasible region $\Pi_0 := \{\pi \mid D^\top \pi \le d\}$ is a nonempty deterministic set.
2. Linear programming duality ensures that a $\pi \in \mathcal{V}_0^k$ satisfies $\pi^\top[\xi(\tilde{\omega}) - C(\tilde{\omega})x] \le h(x, \tilde{\omega})$ almost surely for all $x \in \mathcal{X}$.

Thus, we identify a dual vertex in $\mathcal{V}_0^k$ that provides the best lower bounding function for each summand in the SAA function of (2) as follows:

$$\pi_j^k \in \arg\max\left\{\pi^\top\left[\xi^j - C^j x^k\right] \,\middle|\, \pi \in \mathcal{V}_0^k\right\} \quad \forall \omega^j \in \mathbb{O}^k, j \ne k. \tag{3}$$

Because of the need to associate the dual vertices with an outcome ω^j and the set $\mathcal{V}_0^k$, we use two indices for dual vertex π_j^k. We reserve the superscript (k) to indicate the iteration of the algorithm when the dual vertex is identified. The subscript (j), on the other hand, points to the outcomes in $\mathbb{O}^k$ for which the dual vertex is identified.

During our presentation of the algorithmic aspects, we use the function `argmax(·)` to represent the procedure in (3). Because π_k^k is the optimal dual solution of the subproblem in (1b) with (x^k, ω^k) as input, it can be used to compute the best lower bound for $h(x, \omega^k)$, and hence, invoking the `argmax(·)` procedure is not necessary for $j = k$. These dual vertices $\{\pi_j^k\}$ are used to compute the subgradient β_k^k and the intercept term α_k^k of an affine function $\ell_k^k(x)$ as follows:

$$\alpha_k^k = \frac{1}{k}\sum_{j=1}^{k}\left(\pi_j^k\right)^\top \xi^j; \qquad \beta_k^k = c - \frac{1}{k}\sum_{j=1}^{k}(C^j)^\top \pi_j^k. \tag{4}$$

Similar calculations with respect to the incumbent solution yield another affine function $\hat{\ell}_k^k(x) = \hat{\alpha}_k^k + (\hat{\beta}_k^k)^\top x$, where $\hat{x}^k$ replaces x^k in (3) and the corresponding dual multipliers are used to obtain $(\hat{\alpha}_k^k, \hat{\beta}_k^k)$ in (4). Both the candidate affine function $\ell_k^k(x)$ and the incumbent affine function $\hat{\ell}_k^k(x)$ provide a lower bounding approximation of the SAA function in (2). Note that the SAA function uses a larger collection of outcomes as the algorithm proceeds, and hence, an affine function ℓ_j^j generated at an earlier iteration $j < k$ provides a lower bounding approximation of the SAA function $F^j(x)$, but not necessarily for $F^k(x)$. Consequently, the affine functions generated in earlier iterations need to be adjusted such that they continue to provide lower bound to the current SAA function $F^k(x)$. Under Assumption 3, the adjustment can be achieved using the following recursive calculations:

$$\alpha_j^k \leftarrow \frac{k-1}{k}\alpha_j^{k-1} + \frac{1}{k}L, \qquad \beta_j^k \leftarrow \frac{k-1}{k}\beta_j^{k-1} \quad j = 1, \ldots k-1. \tag{7}$$

Algorithm 1 (Regularized SD Algorithm)

1: **Input:** Scalar parameters $\sigma > 0$, $q > 0$, and $\tau > 0$.
2: **Initialization:** Solve a mean value problem[1] to obtain $x^1 \in \mathcal{X}$. Initialize $\mathbb{O}^0 = \emptyset$, $\mathcal{V}^0 = \emptyset$, $\hat{x}^0 = x^1$, $\sigma^0 = \sigma$, and $k \leftarrow 0$. Set `optimality_flag` to `FALSE`.
3: **while** `optimality_flag` = `FALSE` **do**
4: $k \leftarrow k + 1$.
5: Generate an outcome ω^k by sampling independently from all previous outcomes, and update $\mathbb{O}^k \leftarrow \mathbb{O}^{k-1} \cup \{\omega^k\}$.
6: Setup and solve linear programs to evaluate $h(x^k, \omega^k)$ and $h(\hat{x}^{k-1}, \omega^k)$.
7: Obtain optimal dual solutions π_k^k and $\hat{\pi}_k^k$, and update $\mathcal{V}_0^k \leftarrow \mathcal{V}_0^{k-1} \cup \{\pi_k^k, \hat{\pi}_k^k\}$.
8: Include new affine functions:
9: $\ell_k^k(x) =$ `form_cut`$(\mathcal{V}_0^k, x^k, k)$; $\hat{\ell}_k^k(x) =$ `form_cut`$(\mathcal{V}_0^k, \hat{x}^{k-1}, k)$.
10: Update previously generated affine functions $\{\ell_j^k\}_{\forall j} =$ `update_cuts`$(\{\ell_j^{k-1}\}_{\forall j}, k)$.

11: Update the first-stage objective function approximation as

$$f^k(\boldsymbol{x}) = \max_{j=1,\ldots,k-1} \left\{ \ell_j^k(\boldsymbol{x}), \ell_k^k(\boldsymbol{x}), \hat{\ell}_k^k(\boldsymbol{x}) \right\}. \tag{5}$$

12: $\hat{\boldsymbol{x}}^k$ = `incumbent_update` $(f^k, f^{k-1}, \hat{\boldsymbol{x}}^{k-1}, \boldsymbol{x}^k)$.
13: σ^k = `update_prox`$(\sigma^{k-1}, \|\boldsymbol{x}^k - \hat{\boldsymbol{x}}^{k-1}\|, \boldsymbol{x}^k, \hat{\boldsymbol{x}}^k)$.
14: Obtain the next candidate solution by solving the following regularized master problem:

$$\boldsymbol{x}^{k+1} \in \arg\min \left\{ f^k(\boldsymbol{x}) + \frac{\sigma^k}{2} \left\| \boldsymbol{x} - \hat{\boldsymbol{x}}^k \right\|^2 | \boldsymbol{x} \in \mathscr{X} \right\}. \tag{6}$$

15: `optimality_flag` = `check_optimality`$(\hat{\boldsymbol{x}}^k, \{\ell_j^k(\boldsymbol{x})\})$.
16: **end while**

The most recent piecewise linear approximation $f^k(\boldsymbol{x})$ is defined as the maximum over the new affine functions ($\ell_k^k(\boldsymbol{x})$ and $\hat{\ell}_k^k(\boldsymbol{x})$) and the adjusted old affine functions ($\ell_j^k(\boldsymbol{x})$ for $j < k$). This update is carried out in step 11 of Algorithm 1. The next candidate solution is obtained using this updated function approximation. The cut formation is performed in the function `form_cut(·)`, and the updates are carried out in `update_cuts(·)`.

We postpone the discussion on implementational details until Section 4, and the details regarding subroutines `form_cut(·)`, `update_cuts(·)`, `update_prox(·)`, and `argmax(·)` are in Appendix B. In this paper, we do not discuss the details regarding incumbent update and stopping rules (`incumbent_update(·)` and `check_optimality(·)`, respectively). These details can be found in earlier works, in particular Higle and Sen (1999) and Sen and Liu (2016). We end this section by drawing attention to two salient features of the SD algorithm.

- Dynamic sample size selection: When using the SAA function, a key question is to determine a sample size so that the solution to the sampled instance is an acceptable approximation to the true problem. The works of Shapiro and Homem-de Mello (1998) and Kleywegt et al. (2002) offer some guidelines using measure concentration approaches. Although the theory of SAA recommends a sample size for a given level of accuracy, such sample sizes are known to be conservative, thus prompting a manual trial-and-error strategy of using an increasing sequence of sample sizes. It turns out that the computational demands for such a strategy can easily outstrip computational resources. Perhaps, the best-known computational study applying the SAA function to some standard test instances was reported in Linderoth et al. (2006) in which experiments had to be carried out on a computational grid with hundreds of nodes. The SD algorithm allows for simulation of a new outcome concurrently with the optimization step. This avoids the need to determine the sample size ahead of the optimization step. Further, the SD stopping criteria include both in-sample and out-of-sample procedures to estimate solution quality. These procedures together allow SD to dynamically determine the statistically adequate sample size that can be sufficiently stringent to avoid premature termination.
- Variance reduction: In any iteration, the SD algorithm builds affine lower bounding functions ℓ_k^k and $\hat{\ell}_k^k$, created at the candidate solution $\boldsymbol{x}^k$ and the incumbent solution $\hat{\boldsymbol{x}}^k$, respectively. Both these functions are created using the same stream of outcomes. This notion of using common random numbers is commonly exploited in simulation studies and is known to provide variance reduction in function estimation. In addition to that, Sen and Liu (2016) introduced the concept of "compromise decision" within the SD framework, which uses multiple replications to prescribe a concordant solution across all replications. This replication process reduces both variance and bias in estimation, and therefore, the compromise decision provides more reliable estimates of the optimal value than solutions obtained in individual replications (see theorem 2 of Sen and Liu 2016).

3. SD for Recourse with Random Cost Coefficients

In this section we remove Assumption 4 and allow the cost coefficients of Subproblem (2) to be uncertain. We, however, retain Assumptions 1–3. More specifically, we can think of the random variable $\tilde{\boldsymbol{\omega}}$ as inducing a vector $(\xi^j, \boldsymbol{C}^j, \boldsymbol{d}^j)^\top$ associated with each outcome $\boldsymbol{\omega}^j$. As a result, the dual of Subproblem (1b) with $(\boldsymbol{x}, \boldsymbol{\omega}^j)$ as input is given as

$$\max \left\{ \boldsymbol{\pi}^\top \left[\xi^j - \boldsymbol{C}^j \boldsymbol{x} \right] \middle| \boldsymbol{D}^\top \boldsymbol{\pi} \le \boldsymbol{d}^j \right\}. \tag{8}$$

Notice that the dual polyhedron depends on the outcome $\boldsymbol{d}^j$ associated with observation $\boldsymbol{\omega}^j$. This jeopardizes the dual vertex reutilization scheme adopted by SD to generate the best lower bounding function in (3) as all elements of the set $\mathscr{V}_0^k$ may not be feasible for observation $\boldsymbol{\omega}^j$. The main effort in this section is devoted to designing a decomposition of dual vectors such that the calculations for obtaining lower bounding approximations and for establishing feasibility of dual vectors can be simplified. To do so, we first leverage Assumption 2 to identify a finite collection of basis submatrices of $\boldsymbol{D}$. Because $\boldsymbol{D}$ is deterministic, the basis submatrices are also deterministic. Second, without loss of generality, we represent a random vector as the sum of its deterministic mean and a stochastic shift vector. Consequently, the optimal solution of (8) can be decomposed into a deterministic component and a stochastic shift vector. Although our presentation is focused on general constraint matrix $\boldsymbol{D}$, certain special structured matrices (e.g., total unimodular,

diagonal scaling, etc.) provide ample opportunity for specialized implementation. In the following, we present the general procedure in greater detail.

3.1. Sparsity Preserving Decomposition of Second-Stage Dual Vectors

An important notation used frequently here is the index set of basic variables B^j and a collection of such index sets $\mathcal{B}^k$. When Subproblem (1b) is solved to optimality in iteration k, a corresponding optimal basis is discovered. We record the indices of the basic variables B^k into the collection of previously discovered index sets as $\mathcal{B}^k = \mathcal{B}^{k-1} \cup B^k$. We use $\boldsymbol{D}_B$ to denote the submatrix of $\boldsymbol{D}$ indexed by B.

In the kth iteration, the optimal dual solution $\boldsymbol{\pi}_k^k$ and basis $\boldsymbol{D}_{B^k}$ of the subproblem (1b) with $(\boldsymbol{x}^k, \boldsymbol{\omega}^k)$ as input satisfies

$$\boldsymbol{D}_{B^k}^\top \boldsymbol{\pi}_k^k = \boldsymbol{d}_{B^k}^k. \tag{9}$$

Letting $\bar{\boldsymbol{d}} = \mathbb{E}[\boldsymbol{d}(\tilde{\boldsymbol{\omega}})]$, we can express the kth outcome of cost coefficient in terms of the expected value $\bar{\boldsymbol{d}}$ and deviation from the expectation $\boldsymbol{\delta}(\boldsymbol{\omega}^k)$ (or simply $\boldsymbol{\delta}^k$) as

$$\boldsymbol{d}^k = \bar{\boldsymbol{d}} + \boldsymbol{\delta}^k. \tag{10}$$

Using the decomposed representation of cost coefficients (10) for only the basic variables in (9) yields the optimal dual vector as solution, which has the following form:

$$\boldsymbol{\pi}_k^k = \left(\boldsymbol{D}_{B^k}^\top\right)^{-1} \bar{\boldsymbol{d}}_{B^k} + \left(\boldsymbol{D}_{B^k}^\top\right)^{-1} \boldsymbol{\delta}_{B^k}^k. \tag{11}$$

This allows us to decompose the dual vector into two components $\boldsymbol{\pi}_k^k = \boldsymbol{\nu}^k + \boldsymbol{\theta}_k^k$, where

$$\boldsymbol{\nu}^k = \left(\boldsymbol{D}_{B^k}^\top\right)^{-1} \bar{\boldsymbol{d}}_{B^k}, \qquad \boldsymbol{\theta}_k^k = \left(\boldsymbol{D}_{B^k}^\top\right)^{-1} \boldsymbol{\delta}_{B^k}^k. \tag{12}$$

Although the deterministic component $\boldsymbol{\nu}^k$ is computed only once for each newly discovered basis $\boldsymbol{D}_{B^k}$, the stochastic component $\boldsymbol{\theta}_k^k$ (shift vector) depends on the basis $\boldsymbol{D}_{B^k}$ as well as observation $\boldsymbol{\omega}^k$. Our motivation to decompose the dual vector[2] into these two components rests on the sparsity of deviation vector $\boldsymbol{\delta}_{B^i}^j$ associated with previous ($j = 1, \ldots, k-1$) and future observations $\boldsymbol{\omega}^j, j > i$. Only basic variables with random cost coefficients can potentially have a nonzero element in vector $\boldsymbol{\delta}_{B^i}^j$. When the second stage has a network structure, these calculations are far more streamlined because one can take advantage of the tree structure associated with a basis. Such specializations can significantly reduce both computational effort as well as storage.

3.2. Description of the Dual Vertex Set

Let us now apply the decomposition idea to describe the set of dual vectors associated with an observation $\boldsymbol{\omega}^j$ and all the index sets encountered until iteration k. We denote this set by $\tilde{\mathcal{V}}_j^k$. Because, for every basis $\boldsymbol{D}_{B^i}$, the deterministic component $\boldsymbol{\nu}^i$ is computed only when it is discovered, our main effort in describing the set of dual vertices is in the computation of the stochastic component $\boldsymbol{\theta}_j^i$ (see (12)).

For an index set B^i, let $\tilde{B}^i \subseteq B^i$ index the basic variables whose cost coefficients are random and $\boldsymbol{e}_n$ denote the unit vector with only the nth element equal to one and the remaining elements equal to zero. Using these, we define a submatrix of basis inverse matrix $(\boldsymbol{D}_{B^i}^\top)^{-1}$ as follows:

$$\boldsymbol{\Phi}^i = \left\{ \boldsymbol{\phi}_n^i = \left(\boldsymbol{D}_{B^i}^\top\right)^{-1} \boldsymbol{e}_n \,\middle|\, n \in \tilde{B}^i \right\}. \tag{13}$$

In essence, the matrix $\boldsymbol{\Phi}^i$ is built using columns of the basis inverse matrix corresponding to basic variables with random cost coefficients. We refer to these columns as shifting direction vectors. Using these shifting directions, the stochastic component can be computed as

$$\boldsymbol{\theta}_j^i = \boldsymbol{\Phi}^i \boldsymbol{\delta}_{\tilde{B}^i}^j = \sum_{n \in \tilde{B}^i} \boldsymbol{\phi}_n^i \delta_n^j. \tag{14}$$

With every discovery of a new basis, we compute not only the deterministic component $\boldsymbol{\nu}^i$, but also the shifting vectors $\boldsymbol{\Phi}^i$. These two elements are sufficient to completely represent $\boldsymbol{d}_{B^i}(\boldsymbol{\omega}^j)$ and, consequently, the dual vector $\boldsymbol{\pi}_j^i$ associated with any observation $\boldsymbol{\omega}^j$. Using these, the dual vector set associated with an observation $\boldsymbol{\omega}^j$ can be built as follows:

$$\tilde{\mathcal{V}}_j^k := \left\{ \boldsymbol{\pi}_j^i \,\middle|\, \boldsymbol{\pi}_j^i = \boldsymbol{\nu}^i + \boldsymbol{\Phi}^i \boldsymbol{\delta}_{\tilde{B}^i}^j, \forall i \in \mathcal{B}^k \right\}. \tag{15}$$

With every new observation of $\tilde{\boldsymbol{\omega}}$ encountered, one can use the precomputed elements $\boldsymbol{\nu}^i$ and $\boldsymbol{\Phi}^i$ to generate the set of dual vectors. This limits the calculation to computationally manageable sparse matrix multiplication in (14). We present ideas for efficiently utilizing the decomposition of dual vectors to directly compute coefficients of affine function $\ell(\boldsymbol{x})$ later in Section 4; before that, however, we discuss techniques used to address one additional factor, namely determining feasibility of dual vectors in $\tilde{\mathcal{V}}_j^k$.

3.3. Feasibility of Dual Vectors

We are interested in building the set $\mathcal{V}_j^k \subseteq \tilde{\mathcal{V}}_j^k$ of feasible dual vectors associated with observation $\boldsymbol{\omega}^j$. When Assumption 4 is in place, the cost coefficients are deterministic, and the stochastic shift-vector $\boldsymbol{\delta}^j = 0$ for all observations in $\mathbb{O}^k$. Therefore, every dual vector can be represented using only the deterministic component, that is, $\boldsymbol{\pi}_j^i = \boldsymbol{\nu}^i$ for all j. In other words, for every element in the set $\mathcal{V}_0^k$, there is a unique basis that yields a dual vector $\boldsymbol{\nu}^i = (\boldsymbol{D}_{B^i}^\top)^{-1} \bar{\boldsymbol{d}}_{B^i} = (\boldsymbol{D}_{B^i}^\top)^{-1} \boldsymbol{d}_{B^i}$ that is feasible for all observations in $\mathbb{O}^k$.

In the absence of Assumption 4, a basis $\boldsymbol{D}_{B^i}$ encountered using an observation $\boldsymbol{\omega}^{j'}$ may fail to yield a feasible dual vector for another observation $\boldsymbol{\omega}^j$ for

which $\boldsymbol{\delta}^j \neq \boldsymbol{\delta}^{j'}$. Therefore, we need to recognize whether an index set B^i admits dual feasibility for a given observation $\boldsymbol{\omega}^j$. We formalize the dual feasibility test and necessary computations through the following theorem. In the following, we use the subscript ω for the dual feasible set and the set of dual vertices (Π_ω and $\mathcal{V}_\omega$, respectively) instead of index j when the corresponding statements are true for any outcome $\omega \in \Omega$ and not just for observations $\boldsymbol{\omega}^j \in \mathbb{O}^k$.

Theorem 1. *Let* $\Pi_0 = \{\boldsymbol{\pi} | \boldsymbol{D}^\top \boldsymbol{\pi} \leq \bar{\boldsymbol{d}}\}$ *be a nonempty polyhedral set and* $\mathcal{V}_0$ *be the set of vertices of* Π_0. *For a vector-valued perturbation* $\delta(\omega)$, *let* $\Pi_\omega = \{\boldsymbol{\pi} | \boldsymbol{D}^\top \boldsymbol{\pi} \leq \bar{\boldsymbol{d}} + \boldsymbol{\delta}(\boldsymbol{\omega})\}$ *represent the perturbed polyhedral set and* $\mathcal{V}_\omega$ *the set of vertices of* Π_ω. *Given a feasible index set B associated with a vertex of* Π_0, *the following statements are equivalent:*

1. *A vertex of* Π_0, *say* $\boldsymbol{\nu} \in \mathcal{V}_0$, *can be mapped to a vertex of the perturbed polyhedron,* Π_ω, *as* $\boldsymbol{\nu} + \boldsymbol{\theta}_\omega \in \mathcal{V}_\omega$, *where* $\boldsymbol{\theta}_\omega = (\boldsymbol{D}_B^\top)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega})$ *is a parametric vector of* ω.
2. $\boldsymbol{G}\bar{\boldsymbol{d}} + \boldsymbol{G}\boldsymbol{\delta}(\boldsymbol{\omega}) \geq 0$, *where* $\boldsymbol{G} = [-\boldsymbol{D}_N^\top(\boldsymbol{D}_B^\top)^{-1}, \boldsymbol{I}]$.

Proof. We begin by first showing that 1 $\Rightarrow$ 2.

Because $\boldsymbol{\nu}$ is a vertex of the deterministic polyhedron, Π_0, it is the solution of a system of equations given by $\boldsymbol{D}_B^\top \boldsymbol{\pi} = \bar{\boldsymbol{d}}_B$. That is, $\boldsymbol{\nu} = (\boldsymbol{D}_B^\top)^{-1}\bar{\boldsymbol{d}}_B$. Let N denote the index set of nonbasic variables, and consequently, $\boldsymbol{D}_N$ is the submatrix of the recourse matrix $\boldsymbol{D}$ formed by the columns corresponding to nonbasic variables. Because the current basis $\boldsymbol{D}_B$ is also a basis for perturbated polyhedron Π_ω, a basic solution can be identified by solving the following system of equations:

$$\begin{aligned} & \boldsymbol{D}_B^\top \boldsymbol{\pi} = \bar{\boldsymbol{d}}_B + \boldsymbol{\delta}_B(\boldsymbol{\omega}) = \boldsymbol{D}_B^\top \boldsymbol{\nu} + \boldsymbol{\delta}_B(\boldsymbol{\omega}), \\ \Leftrightarrow \quad & \boldsymbol{D}_B^\top (\boldsymbol{\pi} - \boldsymbol{\nu}) = \boldsymbol{\delta}_B(\boldsymbol{\omega}), \\ \Leftrightarrow \quad & \boldsymbol{\pi} = \boldsymbol{\nu} + \left(\boldsymbol{D}_B^\top\right)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega}). \end{aligned}$$

The second equality is due to $\bar{\boldsymbol{d}}_B = \boldsymbol{D}_B^\top \boldsymbol{\nu}$. Defining $\boldsymbol{\theta}_\omega = (\boldsymbol{D}_B^\top)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega})$, we can obtain a basic solution of perturbed polyhedron as $\boldsymbol{\pi} = \boldsymbol{\nu} + \boldsymbol{\theta}_\omega$. This basic solution is feasible, that is, $\boldsymbol{\pi} \in \mathcal{V}_\omega$, if it also satisfies the following system:

$$\begin{aligned} & \boldsymbol{D}_N^\top \boldsymbol{\pi} \leq \bar{\boldsymbol{d}}_N + \boldsymbol{\delta}_N(\boldsymbol{\omega}) \\ \Leftrightarrow \quad & \boldsymbol{D}_N^\top \boldsymbol{\nu} + \boldsymbol{D}_N^\top \boldsymbol{\theta}_\omega \leq \bar{\boldsymbol{d}}_N + \boldsymbol{\delta}_N(\boldsymbol{\omega}) \\ \Leftrightarrow \quad & 0 \leq \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}\bar{\boldsymbol{d}}_B + \bar{\boldsymbol{d}}_N\right] \\ & \qquad + \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega}) + \boldsymbol{\delta}_N(\boldsymbol{\omega})\right] \\ & \qquad = \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\bar{\boldsymbol{d}}_B^\top, \bar{\boldsymbol{d}}_N^\top\right]^\top \\ & \qquad + \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\boldsymbol{\delta}_B(\boldsymbol{\omega})^\top, \boldsymbol{\delta}_N(\boldsymbol{\omega})^\top\right]^\top \\ & \qquad = \boldsymbol{G}\bar{\boldsymbol{d}} + \boldsymbol{G}\boldsymbol{\delta}(\boldsymbol{\omega}), \end{aligned}$$

where $\boldsymbol{G} = [-\boldsymbol{D}_N^\top(\boldsymbol{D}_B^\top)^{-1}, \boldsymbol{I}]$ with $\boldsymbol{I}$ as an $(n_2 - m_2)$ dimensional identity matrix. Therefore, $\boldsymbol{\pi} = \boldsymbol{\nu} + \boldsymbol{\theta}_\omega \in \mathcal{V}_\omega$ implies that $\boldsymbol{G}\bar{\boldsymbol{d}} + \boldsymbol{G}\boldsymbol{\delta}(\boldsymbol{\omega}) \geq 0$.

In order to show that 2 $\Rightarrow$ 1, we can start with the definition $\boldsymbol{G} = [-\boldsymbol{D}_N^\top(\boldsymbol{D}_B^\top)^{-1}, \boldsymbol{I}]$ and the relation $\boldsymbol{G}(\bar{\boldsymbol{d}} + \boldsymbol{\delta}(\boldsymbol{\omega})) \geq 0$ to obtain the following inequality:

$$\left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\bar{\boldsymbol{d}}_B + \boldsymbol{\delta}_B(\boldsymbol{\omega}), \bar{\boldsymbol{d}}_N + \boldsymbol{\delta}_N(\boldsymbol{\omega})\right]^\top \geq 0.$$

By rearranging the terms, we get

$$\begin{aligned} & \boldsymbol{D}_N^\top\left[\left(\boldsymbol{D}_B^\top\right)^{-1}\bar{\boldsymbol{d}}_B + \left(\boldsymbol{D}_B^\top\right)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega})\right] \\ & \qquad \leq \bar{\boldsymbol{d}}_N + \boldsymbol{\delta}_N(\boldsymbol{\omega}) = \boldsymbol{d}_N. \end{aligned} \tag{16a}$$

Define $\boldsymbol{\pi} = (\boldsymbol{D}_B^\top)^{-1}\bar{\boldsymbol{d}}_B + (\boldsymbol{D}_B^\top)^{-1}\boldsymbol{\delta}_B(\boldsymbol{\omega})$ and note that

$$\begin{aligned} \boldsymbol{D}_B^\top \boldsymbol{\pi} &= \left[\boldsymbol{D}_B^\top \left(\boldsymbol{D}_B^\top\right)^{-1}\right]\bar{\boldsymbol{d}}_B + \left[\boldsymbol{D}_B^\top \left(\boldsymbol{D}_B^\top\right)^{-1}\right]\boldsymbol{\delta}_B(\boldsymbol{\omega}) \\ &= \bar{\boldsymbol{d}}_B + \boldsymbol{\delta}_B(\boldsymbol{\omega}) = \boldsymbol{d}_B. \end{aligned} \tag{16b}$$

From (16a) and (16b), we can conclude that $\boldsymbol{\pi}$ is a vertex of Π_ω. Q.E.D.

The implication of this theorem is that the feasibility of a basis $\boldsymbol{D}_B$ associated with an index set $B \in \mathcal{B}^k$ with respect to an observation $\boldsymbol{\omega} \in \mathbb{O}^k$ can be established by checking if the following inequality is true:

$$\left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\boldsymbol{\delta}_B^\top(\boldsymbol{\omega}), \boldsymbol{\delta}_N^\top(\boldsymbol{\omega})\right]^\top \geq \bar{\boldsymbol{g}}, \tag{17}$$

where $\bar{\boldsymbol{g}} = [\boldsymbol{D}_N^\top(\boldsymbol{D}_B^\top)^{-1}, -\boldsymbol{I}]\bar{\boldsymbol{d}}$. Once again, note that the term on the right-hand side of (17) is a product of a dense matrix and a vector that only needs to be calculated when the basis is discovered for the first time. Moreover, the term $\boldsymbol{D}_N^\top(\boldsymbol{D}_B^\top)^{-1}$ is the submatrix of the tableau formed by the nonbasic variables when the simplex algorithm is employed. The left-hand side term of (17) is a product of a dense matrix and a sparse vector that can be carried out efficiently even for a large number of observations. These calculations are further streamlined based on the position of the variable with random cost coefficient in the basis. In this regard, the following remarks are in order.

Remark 1. Suppose that the index set $\tilde{B}^i = \emptyset$; that is, the basic variables associated with index set $\boldsymbol{B}^i$ have deterministic cost coefficients (equivalently, the variables with random cost coefficients are all nonbasic), then all the elements of $\boldsymbol{\delta}_B(\boldsymbol{\omega})$ are zeros and only $\boldsymbol{\delta}_N(\boldsymbol{\omega})$ has nonzero elements. In such a case, the calculations on the left-hand side of (17) for feasibility check yield

$$\begin{aligned} & \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\boldsymbol{\delta}_B^\top(\boldsymbol{\omega}), \boldsymbol{\delta}_N^\top(\boldsymbol{\omega})\right]^\top \\ & \qquad = \left[-\boldsymbol{D}_N^\top \left(\boldsymbol{D}_B^\top\right)^{-1}, \boldsymbol{I}\right]\left[\boldsymbol{0}^\top, \delta_N^\top(\boldsymbol{\omega})\right]^\top = \boldsymbol{\delta}_N(\boldsymbol{\omega}). \end{aligned} \tag{18}$$

Consequently, we can verify dual feasibility by checking if $\boldsymbol{\delta}_N(\omega) \geq \bar{\boldsymbol{g}}$. Because any feasible outcome lies inside an orthant, this check can be carried out very efficiently.

Remark 2. If $\tilde{B}^i \neq \emptyset$, that is, at least one basic variable associated with index set B^i has random cost coefficient, then we need to explicitly check the feasibility of the

remapped basic solutions using the inequality (17). Nevertheless, one can take advantage of sparsity of these calculations.

Remark 3. The amount of calculations necessary to compute the dual vector and to establish its feasibility depends on the number of random elements in cost coefficient vector $\boldsymbol{d}$. When $\boldsymbol{d}$ is fixed, every basis $\boldsymbol{D}_{B^i}$ is associated with a unique dual vector with stochastic component $\boldsymbol{\theta}_i^j = 0$, which remains feasible for any outcome $\boldsymbol{\omega}^j$. In this case, the calculations reduce to those in the original SD algorithm. When the number of random elements of $\boldsymbol{d}$ is small, the dimension of stochastic component is also small. Consequently, calculations in (12) and (17) impose only a marginal overhead beyond those involved for 2-SLPs with fixed $\boldsymbol{d}$. Therefore, many of the computational advantages of the original SD are retained.

Remark 4. The advantages resulting from low-dimensional randomness in $\boldsymbol{d}$ can also be obtained in some special cases. Suppose that we discover a deterministic matrix $\boldsymbol{\Theta}$ (say of dimension $p_2 \times q_2$) such that $p_2 \ll q_2$, and $\boldsymbol{d}(\boldsymbol{\omega}) = \Theta^\top \boldsymbol{g}(\boldsymbol{\omega})$, with a random vector $\boldsymbol{g}(\boldsymbol{\omega}) \in \mathcal{R}^{p_2}$. Then, we can rewrite $\boldsymbol{d}^\top(\boldsymbol{\omega})\boldsymbol{y} = \boldsymbol{g}(\boldsymbol{\omega})^\top \boldsymbol{\Theta} \boldsymbol{y}$. Such a structure is characteristic of statistical linear factor models that are used in many financial and economic applications (Tsay 2005). By appending a second-stage constraint $z - \boldsymbol{\Theta}\boldsymbol{y} = 0$, we can redefine the second-stage problem with an objective function in the form $\boldsymbol{g}(\boldsymbol{\omega})^\top z$. Here, $\boldsymbol{g}(\boldsymbol{\omega})$ has only $p_2 \ll q_2$ elements, thus achieving the desired reduction in dimension of random elements in cost coefficients. Because the matrix $(\boldsymbol{I}, -\boldsymbol{\Theta})$ does not change with $\tilde{\boldsymbol{\omega}}$, this transformation does not affect the fixed recourse structure required for SD. In other words, scalability of our approach may depend on how the model is constructed, and the modeler is well advised to minimize the number of second-stage variables that have random cost coefficients associated with them.

Remark 5. A very special case arises when one random variable may affect cost coefficients of multiple decision variables simultaneously, that is, $p_2 = 1$. For example, the Federal Reserve's quarterly adjustment of the interest rate might impact returns from multiple investments at the same time. Following an earlier remark, if the new decision variable z is nonbasic, then the high-dimensional orthant reduces to a simple one-dimensional check to see if the random outcome is within a bound. Alternatively, when the variable z is basic, the feasibility check requires us to verify whether the outcome belongs to a certain interval. Recall that $\boldsymbol{\phi}_j = (\boldsymbol{D}_B)^{-1}\boldsymbol{e}_j$, $j \in \tilde{B}$, and in this special case, $|\tilde{B}| = 1$. Hence,

$$\left[-\boldsymbol{D}_N^\top (\boldsymbol{D}_B^\top)^{-1}, \boldsymbol{I}\right]\left[\boldsymbol{\delta}_B^\top(\boldsymbol{\omega}), \boldsymbol{\delta}_N^\top(\boldsymbol{\omega})\right]^\top = -\boldsymbol{D}_N^\top \boldsymbol{\phi}_j \delta_j(\omega) \geq \bar{\boldsymbol{g}}, \quad (19)$$

which implies that $\boldsymbol{\delta}_{lb} \leq \boldsymbol{\delta}_j(\omega) \leq \boldsymbol{\delta}_{ub}$ with

$$\delta_{lb} = \max_{i \in I_1}\left\{\frac{\bar{g}_i}{\left(-\boldsymbol{D}_N^\top \boldsymbol{\phi}_j\right)_i}\right\} \quad \text{where } I_1 = \left\{i \middle| \left(-\boldsymbol{D}_N^\top \boldsymbol{\phi}_j\right)_i > 0\right\}$$

$$\delta_{ub} = \min_{i \in I_2}\left\{\frac{\bar{g}_i}{\left(-\boldsymbol{D}_N^\top \boldsymbol{\phi}_j\right)_i}\right\} \quad \text{where } I_2 = \left\{i \middle| \left(-\boldsymbol{D}_N^\top \boldsymbol{\phi}_j\right)_i < 0\right\}.$$

Note that the subscript i in these equations indicates the ith element (corresponds to a row in basis matrix) of the column vector.

Algorithm 2 (SD for Problems with Random Cost Coefficients)

Steps 1–6 in Algorithm 1.

7a: **if** $\boldsymbol{\omega}^k \notin \mathbb{O}^{k-1}$ (observation generated for the first time) **then** initialize the set $\mathcal{V}_k^{k-1}$ as follows:

$$\mathcal{V}_k^{k-1} \leftarrow \left\{\boldsymbol{\nu}^i + \boldsymbol{\theta}_k^i \,|\, \boldsymbol{G}^i \boldsymbol{\delta}^k \geq \bar{\boldsymbol{g}}^i, \boldsymbol{\theta}_k^i = \boldsymbol{\Phi}^i \boldsymbol{\delta}^k_{\tilde{B}^i}, i \in \mathcal{B}^{k-1}\right\}.$$

7b: **end if**

7c: Obtain the optimal index set B^k, and add it to the collection $\mathcal{B}^k \leftarrow \mathcal{B}^{k-1} \cup \{B^k\}$.

7d: **if** $B^k \notin \mathcal{B}^{k-1}$ (basis encountered for the first time) **then** compute $\boldsymbol{\nu}^k$, $\boldsymbol{\Phi}^k$, $\boldsymbol{G}^k$, and $\bar{\boldsymbol{g}}^k$ as

$$\boldsymbol{\nu}^k = \left(\boldsymbol{D}_{B^k}^\top\right)^{-1} \bar{\boldsymbol{d}}_{B^k} \qquad \boldsymbol{\Phi}^k = \left\{\boldsymbol{\phi}_j^k \middle| j \in \tilde{B}^k\right\}$$
$$\boldsymbol{G}^k = \left[-\boldsymbol{D}_{N^k}^\top \left(\boldsymbol{D}_{B^k}^\top\right)^{-1}, \boldsymbol{I}\right] \quad \bar{\boldsymbol{g}}^k = -\boldsymbol{G}^k \bar{\boldsymbol{d}},$$

where $\boldsymbol{\phi}_j^k$ is defined as in (13).

7e: **for** $\boldsymbol{\omega}^j \in \mathbb{O}^k$ **do** update $\mathcal{V}_j^{k-1}$ as follows:

$$\mathcal{V}_j^k \leftarrow \mathcal{V}_j^{k-1} \cup \left\{\boldsymbol{\nu}^k + \boldsymbol{\theta}_j^k \,\middle|\, \boldsymbol{G}^k \boldsymbol{\delta}^j \geq \bar{\boldsymbol{g}}^k, \boldsymbol{\theta}_j^k = \boldsymbol{\Phi}^k \boldsymbol{\delta}^j_{\tilde{B}^k}\right\}.$$

7f: **end for**

7g: **end if**

8: $\ell_k^k(\boldsymbol{x}) = \texttt{form_cut}(\mathcal{B}^k, \boldsymbol{x}, k)$; $\hat{\ell}_k^k(\boldsymbol{x}) = \texttt{form_cut}(\mathcal{B}^k, \hat{\boldsymbol{x}}, k)$.

Steps 10–16 in Algorithm 1.

3.4. Revised Algorithm Description

Before closing this section, we summarize the modifications to the SD algorithm described in Section 2.1 that are necessary to address 2-SLPs with random cost coefficients. Although bookkeeping in SD of Section 2.1 was restricted to dual vertices, the revision proposed in this section mandates that we store the basis generating the dual vectors. Specifically, the modifications affect steps 7–9 and are summarized in Algorithm 1.

At the beginning of the iteration, we have a collection of index sets $\mathcal{B}^{k-1}$ and observations $\mathbb{O}^{k-1}$. Whenever a new observation $\boldsymbol{\omega}^k$ is encountered, the set of feasible dual vectors $\mathcal{V}_k^{k-1}$ is initialized by computing the stochastic components $\boldsymbol{\theta}_k^i$ using the

parameter $\mathbf{\Phi}^i$ and establishing the feasibility of $\boldsymbol{\pi}_k^i = \boldsymbol{\nu}^i + \boldsymbol{\theta}_k^i$ corresponding to all index sets in $\mathcal{B}^{k-1}$. The feasibility of dual vectors is determined by checking (17), which uses parameters $\boldsymbol{G}^i$ and $\bar{\boldsymbol{g}}^i$. Note that the parameters used for these computations, viz., $\mathbf{\Phi}^i$, $\boldsymbol{\nu}^i$, $\boldsymbol{G}^i$, and $\bar{\boldsymbol{g}}^i$, are precomputed and stored for each index set $B^i \in \mathcal{B}^{k-1}$.

Once the subproblem is solved, we may encounter a new index set B^k. This mandates an update of the sets of feasible dual vectors. In order to carry out these updates, we begin by computing the deterministic parameters $\boldsymbol{\nu}^k$, $\mathbf{\Phi}^k$, $\boldsymbol{G}^k$, and $\bar{\boldsymbol{g}}^k$. These parameters are stored for use not only in the current iteration, but also when new observations are encountered in future iterations. For all observations $\boldsymbol{\omega}^j \in \mathbb{O}^k$, the set of feasible dual vectors is updated with information corresponding to the new basis $\boldsymbol{D}_{B^k}$. This is accomplished by computing the stochastic component $\boldsymbol{\theta}_j^k$, which, in turn, is used to compute the dual vector $\boldsymbol{\pi}_j^k = \boldsymbol{v}^k + \boldsymbol{\theta}_j^k$, and then we include the dual vector in the set $\mathcal{V}_j^{k-1}$ only if it is feasible. This results in an updated set $\mathcal{V}_j^k$.

For all observations $\boldsymbol{\omega}^j (j \neq k)$, the dual vertex that provides the best lower bounding affine function is identified from the corresponding collection of feasible dual vectors $\mathcal{V}_j^k$. This procedure is completed in `form_cut(·)`, which is presented as subroutine 4 in Appendix B. To give a concrete sense of the calculations being carried out, we present an illustrative example in Appendix A. The revised SD algorithm uses the original procedure in Algorithm 1 with steps 7–9 updated as shown in Algorithm 2.

We close this section by noting that the revised algorithm retains the convergence properties of the original SD algorithm with some modifications. Notice that, because of the fixed recourse Assumption 2, there are only finitely many basis matrices $\mathcal{B}$ for the subproblem (1b). Consequently, the set of dual vertices $\tilde{\mathcal{V}}_\omega$ and the set of feasible dual vertices $\mathcal{V}_\omega$ associated with any outcome $\boldsymbol{\omega} \in \Omega$ are also finite. With these observations, the main convergence result for the revised SD algorithm is stated in the following theorem.

Theorem 2. *For a 2-SLP model that satisfies Assumptions 1–3 and $\sigma^k > 0$, the revised SD algorithm generates a sequence $\{\hat{\boldsymbol{x}}^k\}$ that asymptotically converges to an optimal solution with probability one.*

Proof. Under assumption Assumption 3, the dual feasible set Π_ω is nonempty and bounded with probability one. Further, the recourse function is the value of a linear program. These imply that $h(\boldsymbol{x}, \boldsymbol{\omega})$ is a continuous function in $\boldsymbol{x} \in \mathcal{X}$ and bounded. By construction, the collection of index sets satisfies $\mathcal{B}^k \subseteq \mathcal{B}^{k+1} \subseteq \mathcal{B}$. For a given $\boldsymbol{\omega}$, this set of feasible dual vectors $\mathcal{V}_\omega^k$ and $\mathcal{V}_\omega^{k+1}$ built using collections $\mathcal{B}^k$ and $\mathcal{B}^{k+1}$, respectively, satisfy $\mathcal{V}_\omega^k \subseteq \mathcal{V}_\omega^{k+1} \subseteq \mathcal{V}_\omega$. Here $\mathcal{V}_\omega$ is the set of vertices of Π_ω. If we define $h^k(\boldsymbol{x}, \boldsymbol{\omega}) = \max\{\boldsymbol{\pi}^\top(\boldsymbol{\xi}(\boldsymbol{\omega}) - \boldsymbol{C}(\boldsymbol{\omega})\boldsymbol{x}) \mid \boldsymbol{\pi} \in \mathcal{V}_\omega^k\}$, then we have $h^k(\boldsymbol{x}, \boldsymbol{\omega}) \leq h^{k+1}(\boldsymbol{x}, \boldsymbol{\omega}) \leq h(\boldsymbol{x}, \boldsymbol{\omega})$ for all k and for all $(\boldsymbol{x}, \boldsymbol{\omega}) \in \mathcal{X} \times \Omega$. Observing that $\{h^k\}$ is a monotonically increasing sequence of continuous functions bounded from above by the finite function h leads us to the same conclusion as lemma 1 in Higle and Sen (1991). The rest of the proof follows from theorem 5 in Higle and Sen (1994) and theorem 1 in Sen and Liu (2016). Q.E.D.

4. Implementational Details

The SD algorithm is designed to effectively record information that is relevant across different realizations of the random variable and, hence, is shareable. This results in significant computational savings for the SD algorithm. However, one cannot completely take advantage of its features without appropriate implementation of this algorithm. Therefore, our presentation of the algorithm would be incomplete without a detailed discussion of the implementation. In this section, we present the data structures employed in our implementation. Detailed description of the principal subroutines associated with checking feasibility of a dual vector, cut generation, cut updates, and updating the proximal term are given in Appendix B as subroutines 3–6, respectively. The details presented here expand upon those presented in Sen et al. (1994b) and chapter 6 of Higle and Sen (1996).

Our implementation relies on decomposing the information discovered during the course of the algorithm into a deterministic component $\bar{(\cdot)}$ and a stochastic component $\mathbf{\Delta}_{(\cdot)}$, which captures the deviation from the deterministic component. We discussed the value of this decomposition in our calculations of dual vectors described in Section 3.1. Here, we extend this idea to other components of the problem. Note that, for most problems, the number of parameters that are affected by the random variable are significantly smaller than the dimension of the subproblem. Therefore, our decomposition approach reduces the computations necessary by introducing sparsity. Recall that, the random variable affects the right-hand side $\boldsymbol{\xi}$, technology matrix $\boldsymbol{C}$, and the cost coefficient vector $\boldsymbol{d}$. These components can be written as

$$\begin{aligned} \boldsymbol{d}(\boldsymbol{\omega}^j) &= \texttt{d_bar} + \texttt{d_obs[j]},\ \boldsymbol{\xi}(\boldsymbol{\omega}^j) = \texttt{xi_bar} + \texttt{xi_obs[j]}, \\ \boldsymbol{C}(\boldsymbol{\omega}^j) &= \texttt{C_bar} + \texttt{C_obs[j]}, \end{aligned} \tag{20}$$

where the deterministic components `d_bar`, `xi_bar`, and `C_bar` are set to $\bar{\boldsymbol{d}} = \mathbb{E}[\boldsymbol{d}(\tilde{\boldsymbol{\omega}})]$, $\bar{\boldsymbol{\xi}} = \mathbb{E}[\boldsymbol{\xi}(\tilde{\boldsymbol{\omega}})]$, and $\bar{\boldsymbol{C}} = \mathbb{E}[\boldsymbol{C}(\tilde{\boldsymbol{\omega}})]$, respectively. Note that, for deterministic parameters, the stochastic components `*_obs[j]` $= 0$ for all observations $\boldsymbol{\omega}^j$. The strategy to decompose the random elements into deterministic and stochastic

components was also applied in Infanger and Morton (1996) to expedite computation of cut coefficients in a multistage setting. Our choice is driven by the SD data structures for computing coefficients corresponding to newly discovered outcomes (Sen et al. 1994b) and determining feasibility of newly discovered dual vertices. Figure 1 illustrates an information matrix that captures the evolution of information over the course of SD iterations. An element of this information matrix corresponds to an observation–index set pair and contains all terms involving the pair that are necessary for computation during the course of the algorithm. Each iteration of SD may add one observation of $\tilde{\omega}$ and up to two new index sets to the collection $\mathcal{B}^k$ (one from solving the subproblem with the candidate solution as input, and the other from solving with the incumbent solution as input). Calculations associated with each new observation of $\tilde{\omega}$ result in an additional column in the information matrix and discovery of new basis results in a new row. Every basis is associated with an index set $\tilde{B}^i$, which is stored as `basis_idx[i]`.

4.1. Elements of the Information Matrix

These computations are carried out for the lightly shaded cells in the last column and row of the information matrix. There are two sets of elements that are stored in each cell of the information matrix.

The first set is associated with elements that are necessary to establish feasibility of a basis for a given observation ω^j. When a new basis is observed, these elements are necessary to establish feasibility of the basis not only with respect to observations of $\tilde{\omega}$ already encountered, but also for observations that the algorithm may discover in future iterations. To see the development of these elements, notice that the inequality used to establish feasibility of a basis D_{B^i} associated with an index set $B^i \in \mathcal{B}^k$ can be rewritten as

$$G^i\delta - \bar{g}^i = \left[G^i_{\tilde{B}^i}\delta_{\tilde{B}^i}, \delta_{\tilde{N}^i}\right]^\top - \bar{g}^i \geq 0.$$

Here, the calculations only involve columns from the constraint matrix corresponding to basic variables with random cost coefficients (indexed by $\tilde{B}^i$), and constant vector $\bar{g}^i$. These elements are stored as

$$\bar{g}^i = \texttt{sigma_gbar[i]}; \qquad G^i_{\tilde{B}}i = \texttt{lambda_G[i]}. \tag{21}$$

The first term is a floating point vector in $\mathbb{R}^{n_2}$, and the second term is a floating point sparse matrix in $\mathbb{R}^{m_2} \times \mathbb{R}^{q_2}$.

A critical step in computing the lower bounding affine functions in SD is the `argmax(·)` procedure whose mathematical description appears in (3). By introducing the decomposition of subproblem dual vectors into these calculations, we can restate the argument of (3) corresponding to a feasible basis D_{B^i} and observation ω^j as follows:

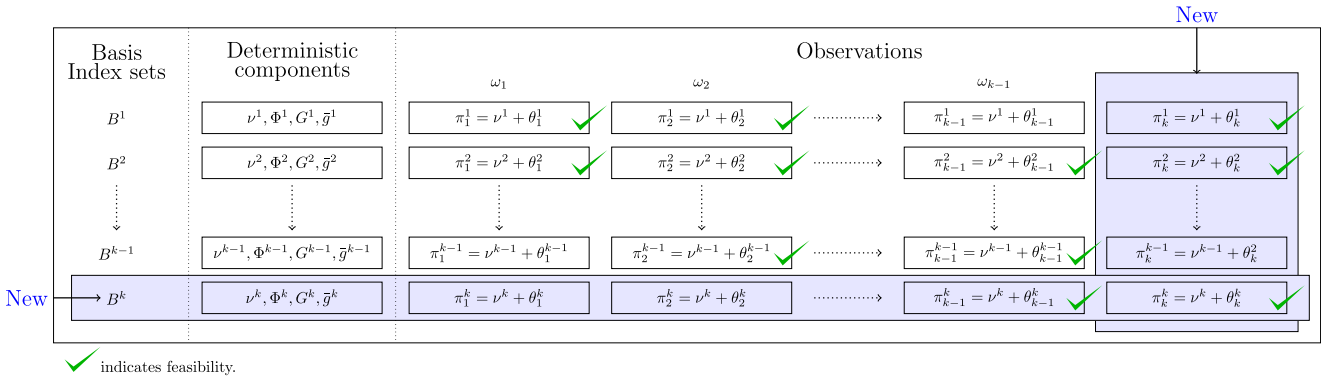
$$\begin{aligned}
&\left(\pi^i_j\right)^\top\left(\xi^j - C^j x\right) \\
&\quad = \left(\nu^i + \Phi^i\delta^j_{\tilde{B}^i}\right)^\top\left[\left(\bar{\xi} + \Delta^j_\xi\right) - \left(\bar{C} + \Delta^j_C\right)x\right] \\
&\quad = \left[\left(\nu^i\right)^\top\bar{\xi} - \left(\nu^i\right)^\top\bar{C}x\right] + \left[\left(\nu^i\right)^\top\Delta^j_\xi - \left(\nu^i\right)^\top\Delta^j_C x\right] \\
&\qquad + \left(\bar{\xi}^\top\Phi^i\right)\delta^j_{\tilde{B}^i} - \left(\bar{C}^\top\Phi^i\right)\delta^j_{\tilde{B}^i}x\Big] + \Big[\left(\Delta^j_\xi\right)^\top\Phi^i\delta^j_{\tilde{B}^i} \\
&\qquad - \left(\Delta^j_C\right)^\top\Phi^i\delta^j_{\tilde{B}^i}x\Big].
\end{aligned} \tag{22}$$

The elements in the first bracketed term are computed only once when the basis is discovered:

$$\left(\nu^i\right)^\top\bar{\xi} = \texttt{sigma_bbar[i][0]}; \;\; \bar{C}^\top\nu^i = \texttt{sigma_Cbar[i][0]}. \tag{23}$$

The first term is a floating point scalar, and the second is a sparse vector in $\mathbb{R}^{n_1}$. To facilitate the calculations in the second bracketed term, note that, if the structures $\xi(\tilde{\omega})$ and $C(\tilde{\omega})$ are such that there are no random elements in row m, then the mth element in the term computes to zero. Thus, we use λ^i_0 to denote a vector obtained by extracting only those components of ν^i that correspond to rows with random elements and naturally stored as a sparse vector `lambda_rho[i][0]`.

Figure 1. (Color online) Illustration of Calculations to Obtain the Set of Feasible Dual Vectors



Recall that $\mathbf{\Phi}^i$ is formed by the columns of the basis matrix corresponding to variables with random cost coefficients (see (13)). We treat these columns in a manner similar to the deterministic component $\boldsymbol{\nu}^i$, and therefore, the terms $\bar{\boldsymbol{\xi}}^\top \mathbf{\Phi}^i$ and $\bar{\boldsymbol{C}}^\top \mathbf{\Phi}^i$ are concatenated to `sigma_bbar` and `sigma_Cbar`, respectively:

$$\begin{aligned} \bar{\boldsymbol{\xi}}^\top \boldsymbol{\phi}_n^i &= \texttt{sigma_bbar[i][n]}; \\ \bar{\boldsymbol{C}}^\top \boldsymbol{\phi}_n^i &= \texttt{sigma_Cbar[i][n]}; \qquad \forall n \in \tilde{B}^i. \end{aligned} \tag{24}$$

This concatenation results in `sigma_bbar[i]` being a floating point vector in $\mathbb{R}^{q_2+1}$ and `sigma_Cbar[i]` a floating point sparse matrix in $\mathbb{R}^{n_2\times(q_2+1)}$. For the calculations in the last bracketed term, we use $\boldsymbol{\lambda}_n^i$ to denote a vector obtained by extracting the components of $\boldsymbol{\phi}_n^i$ corresponding to rows with random elements and store it as a sparse vector `lambda_rho[i][n]`. This results in `lambda_rho[i]` to be a sparse matrix in $\mathbb{R}^{q_2\times(q_2+1)}$.

Given `lambda_rho[i]`, `sigma_bbar[i]`, and `sigma_Cbar[i]`, the computation in (22) can be performed as follows:

$$\begin{aligned} \texttt{delta_b[i][j]} = \texttt{lambda_rho[i][0]} \times \texttt{xi_obs[j]} + \sum_{\texttt{n=1}}^{\texttt{length(basis_idx[i])}} & (\texttt{sigma_bbar[i][n]} \\ & + \texttt{lambda_rho[i][n]} \times \texttt{xi_obs[j]}) \\ & \times \texttt{d_obs_b[i][j]}; \end{aligned} \tag{25a}$$

$$\begin{aligned} \texttt{delta_C[i][j]} = \texttt{lambda_rho[i][0]} \times \texttt{C_obs[j]} + \sum_{\texttt{n=1}}^{\texttt{length(basis_idx[i])}} & (\texttt{sigma_Cbar[i][n]} \\ & + \texttt{lambda_rho[i][n]} \times \texttt{C_obs[j]}) \\ & \times \texttt{d_obs_b[i][j]}; \end{aligned} \tag{25b}$$

where `d_obs_b[i][j]` is defined as a vector with only those elements of `d_obs[j]` indexed by `basis_idx[i]`, that is, $\tilde{\delta}^j_{\tilde{B}_i}$. All matrix multiplications in the preceding calculation are undertaken in the sparse manner. In the preceding notation, `delta_b` in (25a) define scalars, and `delta_C` in (25b) define vectors. Note that, in a deterministic problem, the deviation terms are all zero, and the preceding calculations reduce to zeros as well. The same is case if the observation corresponds to the mean value scenario. Therefore, the terms `delta_b` and `delta_C` are necessary to account for variability in random observations.

The choice of this data structure and calculation is motivated by the fact that `lambda_rho[i]`, `sigma_bbar[i]`, and `sigma_Cbar[i]` require less memory than storing entire vectors $\boldsymbol{\nu}^i$ and $\boldsymbol{\theta}_j^i$. The reason for this is that, in 2-SLPs, the size of the second-stage subproblem is significantly larger than the number of first-stage decisions affecting the subproblem. This, along with the earlier note on the sparsity of deviation of observations from their means, leads to substantial savings in memory usage and required computations. Our implementation takes advantage of further savings achieved by sharing the elements of `lambda_rho[i][n]`, `sigma_bbar[i][n]`, and `sigma_Cbar[i][n]` across the indices in $\mathcal{B}^k$ (this is possible when more than one basis matrix have the same column $\boldsymbol{\phi}_n$).

In summary, the updates to the information matrix are carried out as follows:

1. Column update: Given a new observation $\boldsymbol{\omega}^k$, use (25) to compute the `delta_b[i][k]` and `delta_C[i][k]` for all bases $B^i \in \mathcal{B}^k$.

2. Row update: If a new basis $\boldsymbol{D}_{B^k}$ is seen, then compute the feasibility elements `sigma_gbar[k]` and `lambda_G[k]` in (22). For every element in `basis_idx[k]`, use (24) to compute `sigma_bbar[i][0]` and `sigma_Cbar[i][0]`, and for all $\boldsymbol{\omega}^j \in \mathbb{O}^k$, compute `delta_b[i][j]` and `delta_C[i][j]` using (25).

Because of the dynamic nature of evolution of the information matrix and, therefore, the storage requirement of the SD algorithm, we emphasize the need for careful memory management during implementation of this algorithm.

To conclude, we summarize how the earlier implementation of SD compares with the current version. In 2-SLPs with deterministic cost coefficients, every dual vertex is associated with a unique basis and the index set $\tilde{B}^i = \emptyset$ (`basis_idx` is empty). Therefore, in earlier implementations of the SD algorithm (detailed in Sen et al. 1994b and chapter 6 of Higle and Sen 1996), a cell of the information matrix can be viewed to be associated with a unique dual vertex (instead of a basis) and an observation. Therefore, elements of the cell included `sigma_bbar[i][0]` and `sigma_Cbar[i][0]` in (23) and `delta_b[i][j]` and `delta_C[i][j]` in (25). The dual vertex remains feasible with respect to all observations, and hence, there is no need for the feasibility check procedure. Finally, the calculations in (25) are restricted to only the first terms (without the summation). The original implementation and the updated implementation of the SD algorithm are available under the GNU license from the authors' Github repository as v1.0 and v2.0, respectively (USC3DLAB 2019).

5. Computational Experiments

In this section, we present our computational experience with the SD algorithm when applied to instances with random cost coefficients. Our experiments conducted on instances of three problems compare solutions obtained using the SD algorithm with a collection of solutions obtained by optimizing SAA functions with increasing sample size using the regularized L-shaped method. These experiments

reveal that the SD algorithm (a) provides a higher quality solution for a given computational budget (time) and (b) takes lower computational time to provide a solution of desired quality. We provide a brief description of the problems used in the experiments, the algorithm used to solve the SAA instances, and the experimental setup before presenting the results in detail.

Although there are many challenging instances of 2-SLPs in which randomness affects the right-hand side vector ξ, there is a lack of real-scale instances with random cost coefficients. Although the SG-Portfolio problems (Frauendorfer et al. 1996) have cost coefficients that are modeled as random variables, the available instances have a small set of realizations. Such small sample sizes do not require the use of sampling-based approaches. Therefore, we modified three larger problems from the literature to include random cost coefficients. These problems are `ssn_rc`—a telecommunication network resource allocation problem, `scft`—logistics/supply chain planning problem, and `transship`—a multilocation transshipment problem. Further details regarding these problems and their instances considered for our experiments are given in Appendix C.

Because 2-SLPs are predominantly solved using the L-shaped method, we use it as a benchmark to present our computational results for SD. We begin this section by briefly describing our implementation of the regularized L-shaped method (see Ruszczyński and Shapiro 2003 for details). The L-shaped method is applicable to problems in which uncertainty is characterized by a finite number of realizations (a problem with random variables with discrete distribution or an SAA instance). As before, this set of realizations is denoted by $\mathbb{O}$. In iteration k, we start with a candidate first-stage solution $\boldsymbol{x}^k$ obtained by solving a master program. We solve subproblems for all possible observations $\boldsymbol{\omega}^j \in \mathbb{O}$ and obtain the optimal dual solution $\boldsymbol{\pi}_j^k$. These are used to generate a lower bounding affine function:

$$\alpha_j^k + \boldsymbol{\beta}_j^k x = \boldsymbol{c}^\top \boldsymbol{x} + \sum_{\omega^j \in \mathbb{O}} p^j \cdot (\boldsymbol{\pi}_j^k)^\top [\boldsymbol{\xi}^j - \boldsymbol{C}^j \boldsymbol{x}].$$

In contrast to the coefficients computed in SD (see (4)), which use relative frequency of realizations, these calculations are based on given probabilities associated with the realizations. In this regard, the affine function always provides a "true" lower bound of the first-stage objective function in (1a), or (2) when an SAA function is employed and does not warrant the update mechanism in (7). The algorithm is terminated when the gap between the in-sample upper and lower bound is within an acceptable tolerance (set to a nominal value of 0.001 in our implementation).

Our implementation of the L-shaped method also provides the option of quadratic regularization in the first stage. When regularization is used, the first-stage approximate problem includes a proximal term centered around an incumbent solution, and therefore, the candidate solutions are obtained by solving a quadratic program. The incumbent update rules and the stopping criteria are based on Ruszczyński and Shapiro (2003). This implementation is also available on the authors' Github repository USC3DLAB (2019). Both the regularized L-shaped (RLS) and the SD algorithms are implemented in the C programming language on a 64-bit Intel core i7-4770 CPU at 3.4GHz ×8 machine with 32 GB memory. All linear and quadratic programs were solved using CPLEX 12.8 callable subroutines (CPLEX 2018).

5.1. Experimental Setup

We use an experimental setup similar to that introduced by Mak et al. (1999) in which lower and upper bounds are estimated through multiple replications. For the SAA procedure, we generate M independent samples $\{\boldsymbol{\omega}^{j,m}\}_{j=1}^N$, each of size N. These samples are generated using the standard Monte Carlo simulation and are used to set up the SAA function, which is then solved using the RLS method. Note that variance reduction techniques, such as Latin hypercube sampling, can be employed to reduce variance of the SAA estimates (see Linderoth et al. 2006). The optimal objective function values are denoted as $f_N^{*,m}$ with corresponding solutions as $\boldsymbol{x}_N^{*,m}$. We compute the statistical lower bound to the true optimal value f^* and the $(1-a)$-confidence interval associated with this lower bound as

$$L_N := \frac{1}{M}\sum_{m=1}^{M} f_N^{*,m}; \qquad CI_L = \left[L_N - \frac{z_{a/2} s_L}{\sqrt{M}}, L_N + \frac{z_{a/2} s_L}{\sqrt{M}}\right]. \tag{26}$$

Here, z_a is the $(1-a)$ quantile level of standard normal distribution and s_L is the sample variance of lower bounds $\{f_N^{*,m}\}$.

An upper bound estimate is computed by evaluating the function at solutions obtained from individual replications, that is, $\boldsymbol{x}_N^{*,m}$ for $m = 1, \ldots, M$. The functions are evaluated using a different set of samples $\{\hat{\boldsymbol{\omega}}^{j,m}\}_j$, generated independently from those used for optimization as well as those used in evaluating solutions from other replications. The estimate of the function and the variance associated with it is given by

$$f_{EV,N'}(\boldsymbol{x}) = \frac{1}{N'}\sum_{j=1}^{N'} \left[\boldsymbol{c}^\top \boldsymbol{x} + h(\boldsymbol{x}, \boldsymbol{\omega}^j)\right],$$

$$s_{EV} = \frac{1}{(N'-1)}\sum_{j=1}^{N'} \left[\boldsymbol{c}^\top \boldsymbol{x} + h(\boldsymbol{x}, \boldsymbol{\omega}^j) - f_{EV,N'}(\boldsymbol{x})\right]^2. \tag{27}$$

The estimation is terminated when s_{EV} is within an acceptable threshold ϵ. The final estimation value is denoted as $f_{EV}(\boldsymbol{x})$.

The function estimates at solutions obtained by optimizing the SAA function in different replication can, in turn, be used to obtain an estimate of the upper bound as follows:

$$U_N := \frac{1}{M}\sum_{m=1}^{M} f_{EV}(\boldsymbol{x}_N^{*,m});$$
$$CI_U = \left[U_N - \frac{z_{a/2}s_M}{\sqrt{M}}, U_N + \frac{z_{a/2}s_M}{\sqrt{M}}\right]. \quad (28)$$

We also build a confidence interval around this estimation as shown. Finally we define a pessimistic gap as the difference between the upper limit of CI_U and the lower limit of CI_L. Although the pessimistic gap is a reliable indicator of optimality, in some cases, it can be too demanding to reduce it beyond a certain level. Nonetheless, we use this gap as a measure for solution quality: a lower value indicates higher quality solution. For our experiments with the SAA function, we calculated values $f_N^{*,m}$ for $m = 1,\ldots,M$ and varying number of observations N in the SAA function. The function estimation for computing the upper bound is terminated when $s_{EV}/f_{EV,N'} < \epsilon = 0.01$.

The SD algorithm does not require a preset number of observations, but instead dynamically determines the number of outcomes necessary for obtaining statistically optimal solutions based on the progress made. As in the SAA experiments, optimization is performed over M replications. In each replication, statistical optimality of a solution is established using an in-sample and an out-of-sample stopping rule (see Higle and Sen 1999 and Sen and Liu 2016 for details). These rules basically establish whether the approximations have sufficiently stabilized based on a given tolerance level. In our experiments, we execute all the instances using three increasingly tighter relative tolerances, that is, loose (0.01), nominal (0.001), and tight (0.0001).

In our experiments, we set the number of replications $M = 30$ for both the SAA and SD experiments. We use two sets of M random number seeds; the first set is used to generate the samples to be used in the optimization step, and the second set is used for evaluation.

5.2. Numerical Results

The results from the SAA experiments with varying sample sizes solved using RLS and the SD experiments with varying tolerance values are reported in Tables 1–4. The tables present the average lower bound (26) and upper bound (28) along with half widths of their corresponding 95% confidence intervals (parenthetical values), pessimistic gap as an absolute value and as a fraction of its respective average lower bound (computed as pessimistic gap over average lower bound), and average computational time with its standard deviation.

The `ssn_rc0` instance is the derivative of SSN, which is one of the early 2-SLP models with deterministic cost coefficients. SSN has been used in past to study scalability of sampling-based methods (for example, Linderoth et al. 2006 and Sen and Liu 2016), and therefore, we begin our discussion with results for this model. First, the computational results with RLS reveal that the pessimistic gap can be unacceptably high for small sample SAA functions. In fact, the pessimistic gap does not fall below 10% until one uses $N = 1{,}000$. A comparable solution was obtained using SD with loose tolerance within about 2.3% of the computational time used by RLS.[3]

Across all the instances, the SAA results indicate that the quality of function estimates and solutions improve as the sample size used in creating the SAA function increases. This is clear by noting an overall reducing trend of the pessimistic gap with increasing sample size.[4] These results computationally verify the theoretical properties of the SAA function. However, as the sample size increases, a larger number of subproblems are solved in the second stage during the execution of RLS, thus increasing the computational requirement. This is reflected in the increasing computational time with sample size N.

Our SAA results are comparable in quality (in term of bound estimates) to those reported in Linderoth et al. (2006). The use of a computational grid in the earlier study resulted in a wall clock time of 30–45 minutes per

Table 1. Optimal Objective Value and Optimality Gap Estimates for `ssn_rc0` Instances

Algorithm	Sample size (standard deviation)	Lower bound (95% CI)	Upper bound (95% CI)	Pessimistic gap	Average time, s (standard deviation)
RLS-nominal	50	4.46 (±0.80)	13.63 (±0.36)	10.32 (231.24%)	12.54 (4.41)
	100	6.53 (±0.64)	12.18 (±0.20)	6.49 (99.52%)	31.54 (8.05)
	500	9.06 (±0.39)	10.43 (±0.07)	1.84 (20.30%)	470.34 (208.88)
	1,000	9.56 (±0.25)	10.15 (±0.04)	0.88 (9.17%)	1,698.00 (806.79)
	5,000	9.85 (±0.10)	9.95 (±0.01)	0.22 (2.20%)	30,800.76 (15,187.34)
SD-loose	1,567 (±286)	9.59 (±0.22)	10.21 (±0.05)	0.89 (9.27%)	38.91 (17.60)
SD-nominal	2,315 (±251)	9.72 (±0.13)	10.14 (±0.04)	0.59 (6.03%)	103.86 (30.48)
SD-tight	3,318 (±670)	9.88 (±0.11)	10.12 (±0.04)	0.39 (3.98%)	299.02 (177.00)

Table 2. Optimal Objective Value and Optimality Gap Estimates for `ssn_rcG` ($G = 1, \ldots, 4$) Instances

Algorithm	Sample size (standard deviation)	Lower bound (95% CI)	Upper bound (95% CI)	Pessimistic gap	Average time, s (standard deviation)
			Panel A: `ssn_rc1`		
RLS-nominal	50	38.85 (±8.58)	149.60 (±5.78)	125.11 (321.99%)	21.82 (5.19)
	100	63.62 (±6.48)	129.13 (±3.21)	75.19 (118.20%)	43.69 (5.57)
	500	90.51 (±4.70)	107.31 (±0.91)	22.41 (24.76%)	212.49 (14.96)
	1,000	93.51 (±3.06)	104.56 (±0.40)	14.51 (15.52%)	441.57 (25.41)
	5,000	98.35 (±1.38)	101.70 (±0.14)	4.86 (4.94%)	2421.76 (123.20)
SD-loose	1,600 (±266)	95.64 (±1.95)	104.73 (±0.56)	11.44 (11.96%)	84.31 (31.39)
SD-nominal	2,352 (±410)	97.75 (±1.62)	104.45 (±0.56)	8.72 (8.92%)	221.41 (95.62)
SD-tight	3,305 (±614)	99.30 (±1.55)	103.93 (±0.65)	6.68 (6.73%)	536.48 (242.50)
			Panel B: `ssn_rc2`		
RLS-nominal	50	19.75 (±4.07)	73.40 (±2.59)	60.30 (305.27%)	17.87 (4.40)
	100	30.38 (±3.21)	62.36 (±1.47)	36.67 (120.69%)	36.77 (5.79)
	500	44.92 (±1.88)	51.83 (±0.31)	9.10 (20.26%)	171.06 (9.36)
	1,000	46.77 (±1.02)	50.91 (±0.22)	5.38 (11.50%)	359.77 (22.02)
	5,000	48.29 (±0.56)	49.65 (±0.06)	1.98 (4.09%)	1,904.31 (114.96)
SD-loose	1,759 (±340)	47.54 (±0.81)	51.24 (±0.42)	4.83 (10.39%)	131.80 (74.75)
SD-nominal	2,628 (±423)	48.44 (±0.86)	50.92 (±0.32)	3.58 (7.54%)	366.50 (147.59)
SD-tight	3,939 (±704)	49.02 (±0.59)	50.76 (±0.34)	2.58 (5.44%)	1,018.46 (476.64)
			Panel C: `ssn_rc3`		
RLS-nominal	50	12.55 (±2.37)	48.11 (±2.07)	40.01 (318.85%)	16.47 (2.63)
	100	20.02 (±2.00)	40.65 (±1.17)	23.80 (118.85%)	32.95 (4.18)
	500	29.27 (±1.33)	32.99 (±0.31)	5.36 (18.33%)	152.07 (9.83)
	1,000	29.66 (±0.93)	31.76 (±0.11)	3.15 (10.63%)	298.02 (18.52)
	5,000	30.55 (±0.33)	30.95 (±0.05)	0.77 (2.51%)	1,556.83 (87.75)
SD-loose	1,833 (±334)	31.82 (±0.68)	32.86 (±0.36)	2.08 (6.54%)	140.88 (56.61)
SD-nominal	2,648 (±460)	31.99 (±0.60)	32.59 (±0.26)	1.46 (4.58%)	357.93 (152.25)
SD-tight	3,975 (±662)	32.07 (±0.52)	32.60 (±0.28)	1.32 (4.11%)	1,033.81 (441.54)
			Panel D: `ssn_rc4`		
RLS-nominal	50	8.67 (±1.70)	35.36 (±1.47)	29.87 (344.54%)	14.84 (2.97)
	100	14.86 (±1.44)	30.25 (±1.28)	18.11 (121.85%)	29.06 (4.20)
	500	20.64 (±0.98)	23.89 (±0.14)	4.37 (21.17%)	133.93 (12.03)
	1,000	21.52 (±0.57)	23.24 (±0.11)	2.41 (11.18%)	267.68 (16.34)
	5,000	22.17 (±0.21)	22.56 (±0.04)	0.65 (2.94%)	1,337.37 (94.24)
SD-loose	1,797 (±331)	23.71 (±0.63)	24.64 (±0.35)	1.91 (8.07%)	143.08 (69.65)
SD-nominal	2,798 (±418)	23.99 (±0.43)	24.56 (±0.37)	1.37 (5.70%)	417.94 (156.96)
SD-tight	3,928 (±634)	24.08 (±0.42)	24.50 (±0.37)	1.21 (5.01%)	1,017.31 (428.80)

replication for SAA instances with $N = 5{,}000$. In comparison, our implementation on a single desktop computer takes in excess of 8.5 hours on average to complete the replications. These observations underscore the fact that the SAA approach for challenging 2-SLP models requires significant computational time and/or a high-performance computing environment.

Given that the 2-SLPs are computationally very demanding, the reuse of previously discovered structures of an instance in future iterations provides considerable relief. This is the case in the "sampling-on-the-fly" approach of SD as the results indicate. The SD results for `ssn_rc0` are comparable to those reported in Sen and Liu (2016), which used the previous version of SD implementation. The minor increase in the computational time can be attributed to additional overhead introduced by the data structures necessary to handle the updated information matrix described in Section 4.1. The increasing tightness in tolerance levels used in experiments with the SD algorithm reflect a more demanding requirement in terms of stability of approximation. This results in a higher computational time as indicated in the tables. An increased tolerance level also results in a more accurate estimation of the objective function, and therefore, the solution obtained from such a setting results in a lower pessimistic gap. In any case, our results show that SD provides high-quality solutions in a reasonable amount of time using only ordinary computational resources.

Unlike deterministic optimization, we are more than satisfied to get a solution whose pessimistic gap is within an acceptable limit. This limit is dependent

Table 3. Optimal Objective Value and Optimality Gap Estimates for `scft` Instances

Algorithm	Sample size (standard deviation)	Lower bound (95% CI)	Upper bound (95% CI)	Pessimistic gap	Average time, s (standard deviation)
		Panel A: `scft-6`			
RLS-nominal	50	553,806 (±1,900)	553,536 (±250)	1,879 (0.34%)	0.14 (0.03)
	100	554,129 (±1,659)	553,408 (±226)	1,164 (0.21%)	0.25 (0.04)
	500	554,032 (±518)	553,393 (±220)	100 (0.02%)	0.94 (0.09)
	1,000	553,458 (±325)	553,393 (±220)	480 (0.09%)	1.83 (0.22)
	5,000	553,278 (±134)	553,393 (±220)	469 (0.08%)	8.51 (0.78)
SD-loose	149 (±13)	554,812 (±1,409)	553,598 (±244)	438 (0.08%)	0.20 (0.02)
SD-nominal	274 (±10)	554,391 (±813)	553,499 (±262)	182 (0.03%)	0.40 (0.03)
SD-tight	526 (±5)	554,239 (±541)	553,658 (±269)	230 (0.04%)	0.85 (0.06)
		Panel B: `scft-46`			
RLS-nominal	5	7,391,427 (±120,741)	7,763,122 (±15,146)	507,582 (6.87%)	3,954.39 (905.08)
	10	7,486,263 (±72,180)	7,706,001 (±8,586)	300,503 (4.01%)	6,506.03 (1,003.11)
	25	7,549,045 (±50,789)	7,659,751 (±6,165)	167,661 (2.22%)	10,913.06 (1,217.61)
	50	7,582,958 (±35,836)	7,642,093 (±3,769)	98,740 (1.30%)	14,268.55 (1,298.53)
SD-loose	288 (±28)	7,339,518 (±17,206)	7,648,999 (±3,509)	330,195 (4.50%)	25.24 (4.39)
SD-nominal	430 (±44)	7,408,929 (±13,428)	7,647,755 (±3,828)	256,081 (3.46%)	53.12 (10.41)
SD-tight	1,282 (±295)	7,517,501 (±8,061)	7,642,883 (±4,288)	137,731 (1.83%)	373.78 (158.45)

on the instance, for example, a 5% pessimistic gap may be considered acceptable for ill-conditioned problem instances, such as `ssn_rcG`. As the results indicate, this is already a very high bar to set for SP algorithms. Thus, whenever we achieve 1% or less in pessimistic gap, we consider the instance as being solved. In light of this, it is only appropriate to compare SP algorithms and sample sizes in terms of the pessimistic gap of the solutions reported.

The computational advantage of SD is evident in the results of `ssn_rcG`, `scft`, and `transship` instances. In these instances, the results in the last column of the tables show that the time required for SD to generate a solution of comparable quality is significantly lower than the RLS method. For example, the solution obtained using SD with loose tolerance for ssn_rc0 results in a pessimistic gap of 0.89, which is comparable to the solution obtained using the RLS method applied to a SAA function with sample size of $N = 1{,}000$ (pessimistic gap is 0.88). However, the computational time of SD is lower (by a factor of 43) when compared with the computational time for RLS.

An SAA experiment involves executing multiple optimization steps using RLS, each time increasing the sample size used to build the SAA function if the solution is not acceptable and starting the optimization step from scratch. Therefore, the cumulative experiment time before a solution of desired quality is identified is always significantly higher than the computational time for SD to identify a solution of comparable quality. Moreover, the cumulative time depends on the sequence of sample sizes chosen for experiments (for example, $N = 50, 100, 500, 1{,}000, 5{,}000$ in our experiments for most instances), and there are no clear guidelines for this selection. On the other hand, when SD is executed with a certain tolerance level and if the desired solution quality is not attained, then the tolerance level can be increased and optimization can resume from where it was paused.

Table 4. Optimal Objective Value and Optimality Gap Estimates for `transship` Instances

Algorithm	Sample size (standard deviation)	Lower bound (95% CI)	Upper bound (95% CI)	Pessimistic gap	Average time, s (standard deviation)
RLS-nominal	10	2,333.13 (±3.95)	2,341.36 (±1.60)	13.78 (0.59%)	0.20 (0.02)
	20	2,333.75 (±2.99)	2,338.99 (±1.17)	9.40 (0.40%)	0.31 (0.04)
	30	2,335.65 (±2.10)	2,337.93 (±1.01)	5.38 (0.23%)	0.41 (0.04)
	40	2,335.82 (±1.79)	2,337.69 (±1.00)	4.66 (0.20%)	0.52 (0.04)
	50	2,336.64 (±1.64)	2,337.38 (±0.92)	3.30 (0.14%)	3.84 (2.20)
	75	2,337.38 (±1.35)	2,337.17 (±0.89)	2.02 (0.09%)	0.92 (0.07)
	100	2,337.59 (±0.91)	2,337.01 (±0.85)	1.18 (0.05%)	60.55 (164.51)
SD-loose	168 (±17)	2,336.80 (±0.85)	2,336.88 (±0.92)	1.86 (0.08%)	0.34 (0.06)
SD-nominal	292 (±18)	2,336.44 (±0.83)	2,337.02 (±0.77)	2.18 (0.09%)	0.86 (0.11)
SD-tight	544 (±18)	2,337.00 (±0.59)	2,337.69 (±0.88)	2.16 (0.09%)	2.70 (0.23)

This means that the additional computational time to achieve a higher quality solution (on average) is simply the difference between the computational times reported for different tolerance levels in our tables. Such "on-demand quality" solution is desired in many practical applications.

6. Conclusions

In this paper, we presented the algorithmic enhancements to SD to address fixed recourse 2-SLPs with random right-hand side components as well as random cost coefficients in the second stage. For problems with deterministic cost coefficients, SD reutilizes previously discovered second-stage dual vertices to compute value function approximations in future iterations. However, for problems with random cost coefficients, the feasibility of past dual vertices is not guaranteed. In order to address this issue, we treated the bases that generate the dual vertices as fundamental elements that can be reused in the future. We proposed a sparsity preserving decomposition of dual vectors into deterministic and stochastic components computed using these bases. We presented computationally efficient steps for establishing the feasibility of dual vectors and computing the value function approximations. We also described the data structures employed in our implementation of the algorithm which are designed to be applicable to real-scale problem instances. Our numerical experiments illustrated the computational edge of SD on a variety of test instances when compared with the SAA-based approach.

The multistage stochastic decomposition algorithm was presented in Sen and Zhou (2014), which was the first attempt to extend SD to a multistage setting. Although the authors provide algorithmic details supported by strong analytical results, the computational implementation of this extension faces a critical challenge. When sequential sampling is employed in a multistage setting, the approximations of cost-to-go functions are stochastic in nature. Consequently, the optimization problem at a nonterminal stage has an objective function with random cost coefficients. The enhancements presented in this paper for the two-stage problems provide the foundation necessary to build a successful implementation of the multistage stochastic decomposition algorithm. This will be undertaken in our future research endeavors.

The notion of compromise decision was introduced in Sen and Liu (2016) for sampling-based convex optimization algorithms. The principal idea was to use function approximations and solutions generated during different replication of the algorithm to create a "grand mean" or a compromise problem. The solution obtained by solving this problem, which is termed as the compromise decision, is shown to reduce bias and variance in function estimation and provide a more reliable performance. They also show that the difference between an average solution (computed across all replications) and a compromise decision provides a natural stopping rule. The idea of compromise decisions are best implemented in a parallel environment. This is because all replications (conducted with different seeds for random number generation) should be continued until the stopping criterion, based on the difference between average and compromise decision being small enough, is satisfied. Further bounds on objective function estimates can also be obtained as suggested in Deng and Sen (2018). In any event, because our tests were performed on a serial implementation, we did not adopt the compromise solution. Given the relevance of compromise decisions from the practitioners' point of view, honing the concept of compromise decision for 2-SLPs with random cost coefficients and adapting our implementation for a parallel/high-performance environment will also be an integral part of our future research.

Acknowledgment

The authors thank the referees for their careful review of this paper.

Appendix A. An Illustrative Example

We illustrate the random cost developments via an example that we refer to as the "diamond-shaped" problem. Consider the following problem:

$$\min_{0\le x\le 5} \; f(\boldsymbol{x}) = -0.75x + \mathbb{E}[h(x,\tilde{\boldsymbol{\omega}})], \tag{A.1}$$

where $h(x,\omega)$ is the value of the following LP:

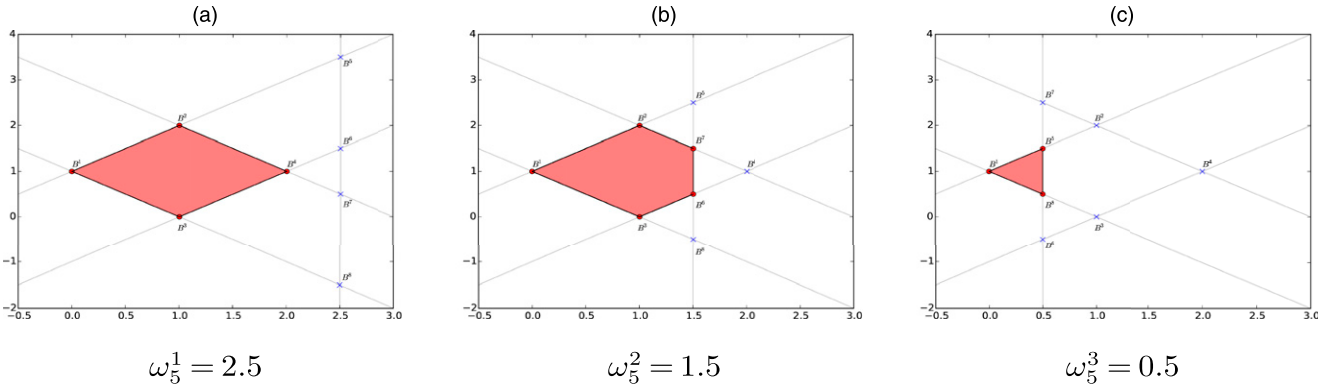
$$\begin{aligned} h(x,\omega) = \min \;& -y_1 + 3y_2 + y_3 + y_4 + \omega_5 y_5 \\ \text{s.t.} \;& -y_1 + y_2 - y_3 + y_4 + y_5 = \omega_6 + \frac{1}{2}x, \\ & -y_1 + y_2 + y_3 - y_4 = \omega_7 + \frac{1}{4}x, \\ & y_i \ge 0,\ i = 1,\dots,5. \end{aligned} \tag{A.2}$$

The random variables $\tilde{\boldsymbol{\omega}}_5, \tilde{\boldsymbol{\omega}}_6$ and $\tilde{\boldsymbol{\omega}}_7$ are assumed to be independent and follow uniform distribution over the intervals $[0,3]$, $[-1,0]$, and $[0,1]$, respectively. The dual LP of the subproblem (A.2) is given by

$$\begin{aligned} h(x,\boldsymbol{\omega}) = \max \;& \left(\omega_6 + \frac{1}{2}x\right)\pi_1 + \left(\omega_7 + \frac{1}{4}x\right)\pi_2 \\ \text{s.t.} \;& -\pi_1 - \pi_2 \le -1,\ \pi_1 + \pi_2 \le 3,\ -\pi_1 + \pi_2 \le 1, \\ & \pi_1 - \pi_2 \le 1,\ \pi_1 \le \omega_5. \end{aligned} \tag{A.3}$$

Notice that the dual feasible region depends on the observation of the random variable $\tilde{\boldsymbol{\omega}}$ through the constraint $\pi_1 \le \omega_5$. We present all the calculations with respect to three observations of the random variable, viz., $\omega_5^1 = 2.5$, $\omega_5^2 = 1.5$,

Figure A.1. (Color online) Basic Solutions and Feasible Regions of Subproblem Dual LP in (A.3)



and $\omega_5^3 = 0.5$. The dual feasible region and the basic solutions of the subproblem dual LP (A.3) with respect to the three observations are shown in Figure A.1. For each basis, we compute $\boldsymbol{\nu}^i$, $\boldsymbol{\Phi}^i$, $\boldsymbol{G}^i$, and $\bar{\boldsymbol{g}}^i$ as given in line 3 of Algorithm 2. Recall that these values are computed on the fly as and when the basis are discovered for the first time. We present these values in Table A.1. In the matrix $\boldsymbol{G}^i$, the columns associated with the variables with random cost coefficients (boldfaced in the table) are sufficient for the feasibility check (see (17)), and it suffices to store only these columns.

When new observations are encountered, the stochastic components required to compute the dual vector are computed, and the feasibility of the resultant dual vector is established based on the inequality (17). These calculations are shows in Table A.2. Note that these computations are based on deviation from the mean of random variable $\boldsymbol{\delta}_5^j = \omega_5^j - \bar{\omega}_5$, that is, $\boldsymbol{\delta}_5^1 = 1.0$, $\boldsymbol{\delta}_5^2 = 0.0$, and $\boldsymbol{\delta}_5^3 = -1.0$.

Table A.1. Bases and Deterministic Components Used to Compute Dual Vectors of Subproblem (A.3)

$i : B^i/N^i$	ν^i	Φ^i	G^i	$\bar{g}^i$
$1 : \{1,3\}/\{2,4,5\}$	$\begin{pmatrix} 0 \\ 1.0 \end{pmatrix}$	$\emptyset$	$\begin{pmatrix} 1.0 & 0 & 1.0 & 0 & \mathbf{0} \\ 0 & 1.0 & 0 & 1.0 & \mathbf{0} \\ 0.5 & 0.5 & 0 & 0 & \mathbf{1.0} \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -2.0 \\ -1.5 \end{pmatrix}$
$2 : \{2,3\}/\{1,4,5\}$	$\begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix}$	$\emptyset$	$\begin{pmatrix} 1.0 & 0 & 1.0 & 0 & \mathbf{0} \\ 0 & 1.0 & 0 & 1.0 & \mathbf{0} \\ -0.5 & 0.5 & 0 & 0 & \mathbf{1.0} \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -2.0 \\ -0.5 \end{pmatrix}$
$3 : \{1,4\}/\{2,3,5\}$	$\begin{pmatrix} 1.0 \\ 0 \end{pmatrix}$	$\emptyset$	$\begin{pmatrix} 1.0 & 0 & 1.0 & 0 & \mathbf{0} \\ 0 & 1.0 & 0 & 1.0 & \mathbf{0} \\ 0.5 & -0.5 & 0 & 0 & \mathbf{1.0} \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -2.0 \\ -0.5 \end{pmatrix}$
$4 : \{2,4\}/\{1,3,5\}$	$\begin{pmatrix} 2.0 \\ 1.0 \end{pmatrix}$	$\emptyset$	$\begin{pmatrix} 1.0 & 0 & 1.0 & 0 & \mathbf{0} \\ 0 & 1.0 & 0 & 1.0 & \mathbf{0} \\ -0.5 & -0.5 & 0 & 0 & \mathbf{1.0} \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -2.0 \\ 0.5 \end{pmatrix}$
$5 : \{3,5\}/\{1,2,4\}$	$\begin{pmatrix} 1.5 \\ 2.5 \end{pmatrix}$	$\left\{\phi_5^5 = \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix}\right\}$	$\begin{pmatrix} 1.0 & \mathbf{2.0} & 1.0 & 0 & 0 \\ -1.0 & \mathbf{-2.0} & 0 & 1.0 & 0 \\ 1.0 & \mathbf{0} & 0 & 0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} -3.0 \\ 1.0 \\ -2.0 \end{pmatrix}$
$6 : \{4,5\}/\{1,2,3\}$	$\begin{pmatrix} 1.5 \\ 0.5 \end{pmatrix}$	$\left\{\phi_5^6 = \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix}\right\}$	$\begin{pmatrix} -1.0 & \mathbf{2.0} & 1.0 & 0 & 0 \\ 1.0 & \mathbf{-2.0} & 0 & 1.0 & 0 \\ 1.0 & \mathbf{0} & 0 & 0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} -1.0 \\ -1.0 \\ -2.0 \end{pmatrix}$
$7 : \{2,5\}/\{1,3,4\}$	$\begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix}$	$\left\{\phi_5^7 = \begin{pmatrix} 1.0 \\ -1.0 \end{pmatrix}\right\}$	$\begin{pmatrix} 1.0 & \mathbf{0} & 1.0 & 0 & 0 \\ -1.0 & \mathbf{2.0} & 0 & 1.0 & 0 \\ 1.0 & \mathbf{-2.0} & 0 & 0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -1.0 \\ -1.0 \end{pmatrix}$
$8 : \{1,5\}/\{2,3,4\}$	$\begin{pmatrix} 1.5 \\ -0.5 \end{pmatrix}$	$\left\{\phi_5^8 = \begin{pmatrix} 1.0 \\ -1.0 \end{pmatrix}\right\}$	$\begin{pmatrix} 1.0 & \mathbf{0} & 1.0 & 0 & 0 \\ 1.0 & \mathbf{2.0} & 0 & 1.0 & 0 \\ -1.0 & \mathbf{-2.0} & 0 & 0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} -2.0 \\ -3.0 \\ 1.0 \end{pmatrix}$

Table A.2. Dual Vectors for All Bases and Their Feasibility Check for $\omega^1 = 2.5$, $\omega^2 = 1.5$, and $\omega^3 = 0.5$

	Observation ω^1 ($\delta_5^1 = 1.0$)				Observation ω^2 ($\delta_5^2 = 0$)				Observation ω^3 ($\delta_5^3 = -1.0$)			
B^i/N_i	π_1^i	$G^i \times \delta^1$	$\bar{g}^i$	Feasibility	π_2^i	$G^i \times \delta^2$	$\bar{g}^i$	Feasibility	π_3^i	$G^i \times \delta^3$	$\bar{g}^i$	Feasibility
B^1/N^1	$\begin{pmatrix}0\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-1.5\end{pmatrix}$	True	$\begin{pmatrix}0\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-1.5\end{pmatrix}$	True	$\begin{pmatrix}0\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\-1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-1.5\end{pmatrix}$	True
B^2/N^2	$\begin{pmatrix}1\\2\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	True	$\begin{pmatrix}1\\2\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	True	$\begin{pmatrix}1\\2\end{pmatrix}$	$\begin{pmatrix}0\\0\\-1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	False
B^3/N^3	$\begin{pmatrix}1\\0\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	True	$\begin{pmatrix}1\\0\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	True	$\begin{pmatrix}1\\0\end{pmatrix}$	$\begin{pmatrix}0\\0\\-1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\-0.5\end{pmatrix}$	False
B^4/N^4	$\begin{pmatrix}2\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\0.5\end{pmatrix}$	True	$\begin{pmatrix}2\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\0.5\end{pmatrix}$	False	$\begin{pmatrix}2\\1\end{pmatrix}$	$\begin{pmatrix}0\\0\\-1\end{pmatrix}$	$\begin{pmatrix}-2\\-2\\0.5\end{pmatrix}$	False
B^5/N^5	$\begin{pmatrix}2.5\\3.5\end{pmatrix}$	$\begin{pmatrix}2\\-2\\0\end{pmatrix}$	$\begin{pmatrix}-3\\1\\-2\end{pmatrix}$	False	$\begin{pmatrix}1.5\\2.5\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-3\\1\\-2\end{pmatrix}$	False	$\begin{pmatrix}0.5\\1.5\end{pmatrix}$	$\begin{pmatrix}-2\\2\\0\end{pmatrix}$	$\begin{pmatrix}-3\\1\\-2\end{pmatrix}$	True
B^6/N^6	$\begin{pmatrix}2.5\\1.5\end{pmatrix}$	$\begin{pmatrix}2\\-2\\0\end{pmatrix}$	$\begin{pmatrix}-1\\-1\\-2\end{pmatrix}$	False	$\begin{pmatrix}1.5\\0.5\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-1\\-1\\-2\end{pmatrix}$	True	$\begin{pmatrix}0.5\\-0.5\end{pmatrix}$	$\begin{pmatrix}-2\\2\\0\end{pmatrix}$	$\begin{pmatrix}-1\\-1\\-2\end{pmatrix}$	False
B^7/N^7	$\begin{pmatrix}2.5\\0.5\end{pmatrix}$	$\begin{pmatrix}0\\2\\-2\end{pmatrix}$	$\begin{pmatrix}-2\\-1\\-1\end{pmatrix}$	False	$\begin{pmatrix}1.5\\1.5\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-1\\-1\end{pmatrix}$	True	$\begin{pmatrix}0.5\\2.5\end{pmatrix}$	$\begin{pmatrix}0\\-2\\2\end{pmatrix}$	$\begin{pmatrix}-2\\-1\\-1\end{pmatrix}$	False
B^8/N^8	$\begin{pmatrix}2.5\\-1.5\end{pmatrix}$	$\begin{pmatrix}0\\2\\-2\end{pmatrix}$	$\begin{pmatrix}-2\\-3\\1\end{pmatrix}$	False	$\begin{pmatrix}1.5\\-0.5\end{pmatrix}$	$\begin{pmatrix}0\\0\\0\end{pmatrix}$	$\begin{pmatrix}-2\\-3\\1\end{pmatrix}$	False	$\begin{pmatrix}0.5\\0.5\end{pmatrix}$	$\begin{pmatrix}0\\-2\\2\end{pmatrix}$	$\begin{pmatrix}-2\\-3\\1\end{pmatrix}$	True

Appendix B. Principal SD Subroutines

This section presents SD subroutines for checking feasibility, computing cut coefficients, and updating the cuts and the proximal parameter.

B.1. Checking Feasibility

Feasibility of a given basis $\boldsymbol{D}_{B^i}$ for an observation $\boldsymbol{\omega}^j$ is verified using the inequality in (17). The steps are presented as function `check_feasibility(·)` in subroutine 3. The process of checking feasibility is completed in two steps. The first one is a trivial check for nonbasic variables: $\boldsymbol{\delta}^j_{\bar{N}^i} - \bar{\boldsymbol{g}}^i_{\bar{N}^i} \geq 0$. The second step is for basic variables, which involves checking the inequality $\boldsymbol{G}^i_{\bar{B}^i}\boldsymbol{\delta}_{\bar{B}^i} - \bar{\boldsymbol{g}}^i_{\bar{B}^i} \geq 0$. Once the feasibility check is completed, the results (the return values of `check_feasibility(·)`) are stored in a Boolean vector `obsFeasibility[i]`, one for each index set $\boldsymbol{B}^i \in \mathcal{B}^k$. The next set of calculations is carried out only if the basis is found to be feasible. The use of this Boolean vector avoids the need to store separate sets of feasible dual vectors $\mathcal{V}^k_j$ for each observation ω^j.

B.1.1. Subroutine 3: Subroutine to Check Feasibility of a Dual Vector.

```
1: function check_feasibility(lambda_G, sigma_gbar,
   d_obs, basis_idx, i, j)
2:   feasFlag = TRUE; n = 1
3:   while n <= length(basis_idx) && feasFlag == TRUE do
4:     if n ∉ basis_idx then
5:       feasFlag = d_obs[j][n] - sigma_gbar[i][n] >= 0;
6:     else
7:       feasFlag = lambda_G[i][n] × d_obs[j][n] - sigma_
           gbar[i][n] >= 0;
8:     end if
9:     n = n + 1
10:  end while
11:  return feasFlag
12: end function
```

B.2. Computing and Updating Cut Coefficients

With the elements of the information matrix defined, we are now in a position to present the calculations involved in computing the coefficients of the lower bounding affine functions. The principal step is the `argmax(·)` procedure presented in subroutine 4, which is invoked once for each observation $\boldsymbol{\omega}^j \in \mathbb{O}^k$. The argument for this operation in (3) is computed using elements of the information matrix as

$$(\boldsymbol{\pi}^i_j)^\top(\boldsymbol{\xi}^j - \boldsymbol{C}^j\boldsymbol{x}) = \texttt{sigma_bbar[i][0]} + \texttt{delta_b[i][j]} - (\texttt{sigma_Cbar[i][0]} + \texttt{delta_C[i][j]}) \times \texttt{x}.$$

Note that this calculation is performed only for index set $\boldsymbol{B}^i \in \mathcal{B}^k$, which yields a feasible basis with respect to the observation $\boldsymbol{\omega}^j$. The output of the argmax procedure is the coefficients alpha_star and beta_star. These coefficients correspond to the intercept and subgradient of the affine function, which provides the best lower bound to $h(\boldsymbol{x}, \boldsymbol{\omega}^j)$. The `argmax(·)` also returns a pointer to an index set in $\mathcal{B}^k$, which is used to compute these coefficients. These pointers are stored in an integer vector called `i_star` for their use in resampling the cut for our statistical optimality tests.

We define a cut data structure with fields `alpha`, `beta`, `sample_size`, and `i_star`, where `sample_size` denotes the number of observations that are used to create the cut (this is the same as iteration count k as one sample is added every iteration). The computation of the SD affine functions is shown in function `form_cut(·)` in subroutine 4.

B.2.1. Subroutine 4: Subroutines to Form Cuts.

```
1: function argmax(bases,x,j)
2:   maxval = -∞;
3:   for i = 1...length(bases) do
4:     if obsFeasibility[i][j] = = TRUE then
5:       val = sigma_bbar[i][0] + delta_b[i][j]
             - (sigma_Cbar[i][0] + delta_C[i][j]) × x;
6:       if maxval < val then
7:         alpha_star = sigma_bbar[i][0] + delta_b[i][j]
8:         beta_star = -(sigma_Cbar[i][0] + delta_C[i][j]);
9:         maxval = val;
10:        i_star = i;
11:      end if
12:    end if
13:  end for
14:  return (alpha_star, beta_star, i_star);
15: end function
```

```
16: function form_cut(bases,x,k)
17:   /* alpha, beta, sample_size, i_star and eta_coeff
        are fields of cut. */
18:  alpha = 0, beta = [0,...,0], sample_size = k, eta_coeff = 1.0;
19:  for j = 1,...,k do
20:    (alpha_star, beta_star, i_star[j]) = k
         argmax(bases,x,j);
21:    alpha = (j-1)/j × alpha + alpha_star/j;
22:    beta = (j-1)/j × beta + beta_star/j;
23:  end for
24:  return cut;
25: end function
```

B.2.2. Subroutine 5: Subroutine to Update Cuts.

```
1: function update_cut(cuts, k)
2:   for c = 1 ... length(cuts) do
3:     eta_coeff[c] = k/(k-1) × eta_coeff[c];
4:     if L ≠ 0 then
5:       rhs[c] = rhs[c] + (k-cut_iter[c])/cut_iter[c]) L;
6:     end if
7:   end for
8:   return cuts;
9: end function
```

B.3. Updating the Proximal Parameter

The SD algorithm uses a proximal term built around the incumbent solution $\hat{x}^k$ to form the regularized master program. Our implementation includes certain enhancements that involve the proximal parameter σ^k and are designed to improve the computational performance of the SD algorithm. Instead of keeping the proximal parameter constant, we allow it to be updated throughout the course of the algorithm (hence, we use iteration index in our notation σ^k). In general, the σ^k is decreased when the incumbent changes and $\|x^{k+1} - \hat{x}^k\|$ increases from one iteration to the next and increased when the incumbent does not change. This procedure is presented as function `update_prox(·)` in subroutine 4 in which σ^k is referred to as `prox_param`. We note that this procedure does not alter the theoretical properties of the regularized SD algorithm because $\sigma^k > 0$ for all k.

B.3.1. Subroutine 6: Subroutine to Update Proximal Parameter.

```
1: /* r1, r2 < 1.0 are given constants, norm(·) computes
      two-norm. */
2: function update_prox(prox_param, old_norm, x, hat_x,
     new_incumb)
3:   if new_incumb = = TRUE then
4:     new_norm = norm(x, hat_x);
5:     if new_norm ≥ r1 × old_norm then
6:       prox_param = min(max(r2 × prox_param,
           pow(10,-6)), 1.0);
7:     end if
8:   else
9:     prox_param = prox_param/r2;
10:  end if
11:  return prox_param;
12: end function
```

Recall that, as the algorithm progresses, the coefficients of the lower bounding affine function are updated as shown in (7). The affine functions are added as constraints in the master problem. The direct implementation of this procedure requires updating all the coefficients $(\alpha_j^{k-1}, \boldsymbol{\beta}_j^{k-1})$ of the affine functions. Computationally, this may prove to be cumbersome. However, one can view these updates in the mathematically equivalent form for $j = 1, \ldots, k-1$ as

$$\begin{aligned}\eta &\geq \alpha_j^k + \left(\boldsymbol{\beta}_j^k\right)^\top x = \frac{k-1}{k}\left(\alpha_j^{k-1} + \left(\boldsymbol{\beta}_j^{k-1}\right)^\top x\right)\\ &= \frac{j}{k}\left(\alpha_j^j + \left(\boldsymbol{\beta}_j^j\right)^\top x\right) + \frac{k-j}{k}L\\ &\equiv \left(\frac{k}{j}\right)\eta - \left(\boldsymbol{\beta}_j^j\right)^\top x \geq \alpha_j^j + \frac{k-j}{j}L.\end{aligned}$$

This representation allows us to retain most of the coefficients at values obtained when they were computed (for example, $\boldsymbol{\beta}_j^j$, which were computed in iteration j). The updates are restricted only to a single column corresponding to the auxiliary variable η. The right-hand side is updated only if $L \neq 0$. These updates are carried out in the function `update_cut(·)` in subroutine 4 in which `cut_iter[·]` encodes the iteration when a cut is generated (i.e., j).

Appendix C. Test Instances

In this section, we provide a brief description of the test instances used in our computational study.[5] The size of the problem instances is summarized in Table C.1. The fourth column corresponds to the number of second-stage parameters affected by the random variable (RV) presented as number of right-hand side elements and objective cost coefficients. These instances are available in the SMPS format in the authors' Github repository USC3DLAB (2019).

C.1. Sonet-Switched Network with Random Cost Coefficients

This problem has its origin in telecommunication network planning and was presented in the original form in Sen et al. (1994a). The problem considers allocation of additional

Table C.1. Details of Problem Instances

Instance name	First-stage variables/constraints	Second-stage variables/constraints	Number of RVRHS + Obj.	Magnitude of outcomes
`ssn_rcG`	89/1	709/178	86 + G	10^{70}
`SCFT-6`	30/18	86/36	12 + 2	10^{9}
`SCFT-46`	602/174	348/1,480	184 + 2	10^{129}
`transship`	7/0	77/35	7 + 7	Normal distribution

capacity to the links in a network that provides private line telecommunication services. These service requests arise between pairs of nodes in the network. In the original model, the service quantity (demand) is the only parameter assumed to depend on the random variable, and the goal is to allocate capacity so as to reduce the total demand lost. Here we extend the model by associating a cost with lost demand, which also depends on the random variable. We briefly describe the model here.

We denote the communication links in the network by $\mathcal{L}$. A service request is characterized by a pair of origin–destination nodes, which is assumed to be an element of the set $\mathcal{S}$. Each service request (say i) can be satisfied over a set of routes denoted by $\mathcal{R}_i$. If there are no routes with sufficient capacity available to accommodate the request, it cannot be fully served. The existing capacity of the links denoted as e_ℓ needs to be appended by additional capacity x_ℓ which constitutes the first-stage decision variable. The total amount of capacity that can be added is restricted to be with a budget denoted as b. This results in following first-stage optimization problem:

$$\min \left\{ \mathbb{E}[h(\boldsymbol{x}, \tilde{\boldsymbol{\omega}})] \,\middle|\, \sum_{\ell \in \mathcal{L}} x_\ell \le b,\ x_\ell \ge 0\ \forall \ell \in \mathcal{L} \right\}.$$

Once the capacity decisions have been made, requests for service can be routed in a manner that efficiently utilizes the network resources. To this effect, the function $h(x, \omega)$ captures the cost of unsatisfied service and is the value of the following LP:

$$\begin{aligned} \min\ & \sum_{g \in \mathcal{G}} d_g(\boldsymbol{\omega}) \left[\sum_{i \in \mathcal{S}_g} u_i \right] \\ \text{s.t.}\ & \sum_{i \in \mathcal{S}} \sum_{i \in \mathcal{R}_i} \boldsymbol{A}_{ir} f_{ir} \le e_\ell + x_\ell \qquad \forall \ell \in \mathcal{L} \\ & \sum_{r \in \mathcal{R}_i} f_{ir} + u_i = \xi_i(\boldsymbol{\omega}) \qquad \forall i \in \mathcal{S}, \\ & f_{ir}, u_i \ge 0 \qquad \forall r \in \mathcal{R}_i, i \in \mathcal{S}. \end{aligned}$$

Here the decision variable u_i captures the number of units of unsatisfied demand, and f_{ir} denotes the amount of service satisfied via route $r \in \mathcal{R}_i$ for every service request $i \in \mathcal{S}$. The term $\boldsymbol{A}_{ir}$ denotes an incidence matrix whose element-ℓ is one if link-ℓ belongs to route $r \in \mathcal{R}_i$ and zero otherwise. Although the first set of constraints ensures that the total flow on each link in the network does not exceed the sum of existing and newly installed capacity, the second set of constraints accounts for the total amount of service requested by $i \in \mathcal{S}$ as either satisfied over all the associated routes or as unsatisfied. The right-hand side of the second set of constraints, representing the total service requested, depends on the random variable. We distinguish the service requests by clustering them into groups denoted by the set $\mathcal{G}$ and penalize the unsatisfied service request differently across these groups. Moreover, the penalty cost for group g, denoted by d_g, also depends on the random variable. Note that the original model can be recovered by setting $|\mathcal{G}| = 1$ and $d_g(\tilde{\boldsymbol{\omega}}) = 1$, almost surely.

To create the instances of this model, we assume that the demand and the cost coefficients are independent of one another. We use the original stochastic information for the service demand data and use a discrete distribution to model the demand random variable. We created different instances by varying the number of groups. These instances are denoted as `ssn_rcG`, where $G = 1, \ldots, 4$.

C.2. Supply Chain–Freight Transportation Optimization

This problem is aimed at mi-term planning of a global multiproduct chemical supply chain and appeared first in You et al. (2009). A set of products are manufactured and distributed through a given global supply chain that includes a large number of worldwide customers and a number of geographically distributed plants and distribution centers. The facilities can hold inventory and are connected to each other by transportation links. The problem is to determine the monthly production and inventory levels of each facility and the monthly shipping quantities between network nodes such that the total expected cost of operating the global supply chain is minimized while satisfying customer demands over the specified planning horizon. Because of involved notations in the model, we restrict to a verbal presentation of the model and refer the reader to the original publication for the mathematical formulation.

The problem involves decisions made over a finite horizon at multiple discrete time epochs. The inventory, transshipment, and production decisions are made at the beginning of the time period. These decisions are subject to mass balance constraints at the plants, distribution centers, and customer locations. These mass balance equations take into account the transportation time during the shipping process such that the freight can arrive at the destination on time. The initial inventory required for mass balance at all facilities is assumed to be known, and an explicit constraint ensures that the inventory level is maintained above a certain minimum requirement. In addition to these constraints, the decision variables are bounded by their respective capacities. The decisions for the first time period are made in a here-and-now manner, and the decisions for the remaining horizon are adaptive in nature and depend on the realization of random variables corresponding to customer demands and freight rates. The objective function is to minimize the total expected cost, which includes the inventory holding costs at facilities, freight and

facility throughput costs for shipment between facilities (including customer locations), and penalty costs for unmet demand.

Unfortunately, the data used in the original publication is not publicly available. Therefore, we have generated two instances of the model using synthetic data. The first instance (`SCFT-6`) is for a supply chain network with two plants, three distribution centers, and six customer locations, which are connected by 14 transportation links. The second, more realistic, instance is comparable in size to the to that used in You et al. (2009). This instance has five plants, 13 distribution centers, and 46 customer locations. These facilities are connected by 131 transportation links. In our instances, the model horizon is set to three months; the first month is considered to be the here-and-now stage, and the remaining two months constitute the second stage. Further, in our instances, 10% of the freight rates are affected by randomness and are assumed to follow a discrete distribution. The size of the two problem instances is summarized in Table C.1.

C.3. Multilocation Transshipment Problem

This problem, adopted from Herer et al. (2006), involves several nonidentical retailers and one supplier that coordinate their replenishment strategies and transshipments to minimize the long-run expected cost. The system inventory is periodically reviewed, and the replenishment orders are placed with the supplier. At the beginning of a period, replenishment orders (placed in the previous time period) arrive, and they are used to satisfy any outstanding backlog and to increase the inventory level. Following this, the demand is realized, transshipments are completed immediately and are used to satisfy the demand, the unsatisfied demand is backlogged, and the inventory quantities are updated. Finally, based on the inventory and backlog quantities, new replenishment decisions are placed with the supplier. The system costs are determined by transshipment costs, inventory holding cost, and backlog penalties. In the original work, the authors assume that only the demand at each retailer in a period is random and stationary over time. We extend their model by also allowing the transshipment costs to be random.

Undera non–shortage-inducing policy assumption, Herer et al. (2006) show that there exists an optimal order-up-to policy. The order-up-to quantities are estimated using a sample-path optimization method, such as infinitesimal perturbation analysis. Alternatively, one can formulate this problem as a 2-SLP with the first stage tasked with identifying the order-up-to quantities and the second stage associated with replenishment orders and transshipments. To present the 2-SLP formulation, we use N to denote the number of suppliers, s_i and $d_i(\tilde{\omega})$ to denote the first-stage decision representing the order-up-to quantity, and random variable dependent demand at retailer i. The second-stage decisions are the ending inventory e_i, stock used to satisfy demand f_i, inventory increased through replenishment q_i, and amount of backlog met after replenishment r_i at retailer i and stock at retailer i used to meet demand at retailer j using transshipment t_{ij}. The unit holding and backlog penalty costs at retailer i, unit transshipment cost from retailer i to j are denoted as h_i, p_i, and $c_{ij}(\tilde{\omega})$, respectively. Given an order-up-to policy for the replenishment quantities, Herer et al. (2006) suggest a linear cost network flow model to identify the optimal transshipment quantities. We use the same approach to state the 2-SLP as follows:

$$\min_{s_i \geq 0,\ i=1,\ldots,N} \mathbb{E}[h((s_i)_i, \tilde{\omega})], \tag{C.1}$$

where

$$\begin{aligned} h((s_i)_i, \tilde{\omega}) = \min \quad & \sum_{i=1}^{N} (h_i e_i + p_i r_i) + \sum_{i=1, i\neq j}^{N} c_{ij}(\tilde{\omega}) t_{ij} \\ \text{s.t.} \quad & f_i + \sum_{j=1, j\neq i}^{N} t_{ij} + e_i = s_i \quad i = 1, \ldots, N; \\ & f_i + \sum_{j=1, j\neq i}^{N} t_{ij} + r_i = d_i(\tilde{\omega}) \quad i = 1, \ldots, N; \\ & \sum_{i=1}^{N} (r_i + q_i) = \sum_{i=1}^{N} d_i(\tilde{\omega}); \\ & e_i + q_i = s_i \quad i = 1, \ldots, N; \\ & e_i, f_i, q_i, r_i, t_{ij} \geq 0 \quad i, j = 1, \ldots, N. \end{aligned} \tag{C.2}$$

To build our instances with random cost coefficients, we have used the demand distribution information presented in table 3 of Herer et al. (2006). The cost coefficients are nonidentical and built as described in section 4.5 of Herer et al. (2006). We assume that the transshipment costs follow a normal distribution with mean equal to the deterministic quantities from the reference and standard deviation set to 20% of the mean. This instance is named as `transship` in our database.

Endnotes

[1] The mean or expected value problem is a deterministic optimization problem obtained by replacing all random variables in (1) by their expected values.

[2] The dual vector $\boldsymbol{\pi}_j^i$ and its components $(\boldsymbol{v}^i, \boldsymbol{\theta}_j^i)$ are indexed by j and i. The subscript j denotes the observation $\boldsymbol{\omega}^j$, and the superscript i denotes the index set $\boldsymbol{B}^i$ associated with the dual vector.

[3] In certain cases—for example, transship instances with $N \geq 100$—a few replications of the RLS experiments take significantly longer than the others. This results in the large standard deviation in the average computational time as seen for $N = 100$ in Table 4.

[4] Because these are stochastic estimates that depend on randomly sampled quantities, one should not expect a strictly monotonic behavior. Moreover, the numbers reported are averages across multiple replications.

[5] The scope of the variables and parameters defined here is restricted to individual subsections.

References

CPLEX (2018) *CPLEX Callable Library (C API) Reference Manual*, 12.8 ed. (IBM, Armonk, NY).

Deng Y, Sen S (2018) *Learning Enabled Optimization: Toward a Fusion of Statistical Learning and Stochastic Programming*. Technical report, University of Southern California, Los Angeles.

Frauendorfer K, Härtel F, Reiff MF, Schürle M (1996) *SG-Portfolio Test Problems for Stochastic Multistage Linear Programming* (Springer, Berlin, Heidelberg).

Gangammanavar H, Sen S, Zavala VM (2016) Stochastic optimization of sub-hourly economic dispatch with wind energy. *IEEE Trans. Power Systems* 31(2):949–959.

Herer YT, Tzur M, Yücesan E (2006) The multilocation transshipment problem. *IIE Trans.* 38(3):185–200.

Higle JL, Sen S (1991) Stochastic decomposition: An algorithm for two-stage linear programs with recourse. *Math. Oper. Res.* 16(3):650–669.

Higle JL, Sen S (1994) Finite master programs in regularized stochastic decomposition. *Math. Programming* 67(1–3):143–168.

Higle JL, Sen S (1996) *Stochastic Decomposition: A Statistical Method for Large Scale Stochastic Linear Programming* (Kluwer Academic Publishers, Boston).

Higle JL, Sen S (1999) Statistical approximations for stochastic linear programming problems. *Ann. Oper. Res.* 85:173–193.

Infanger G, Morton DP (1996) Cut sharing for multistage stochastic linear programs with interstage dependency. *Math. Programming* 75(2):241–256.

Kleywegt AJ, Shapiro S, Homem-de Mello T (2002) The sample average approximation method for stochastic discrete optimization. *SIAM J. Optim.* 12(2):479–502.

Linderoth J, Shapiro A, Wright S (2006) The empirical behavior of sampling methods for stochastic programming. *Ann. Oper. Res.* 142(1):215–241.

Mak W, Morton DP, Wood K (1999) Monte Carlo bounding techniques for determining solution quality in stochastic programs. *Oper. Res. Lett.* 24(1):47–56.

Ruszczyński A, Shapiro A (2003) *Handbooks in Operations Research and Management Science: Stochastic Programming*, vol. 10 (Elsevier, Amsterdam).

Sen S, Liu Y (2016) Mitigating uncertainty via compromise decisions in two-stage stochastic linear programming: Variance reduction. *Oper. Res.* 64(6):1422–1437.

Sen S, Zhou Z (2014) Multistage stochastic decomposition: A bridge between stochastic programming and approximate dynamic programming. *SIAM J. Optim.* 24(1):127–153.

Sen S, Doverspike RD, Cosares S (1994a) Network planning with random demand. *Telecommunication Systems* 3(1):11–30.

Sen S, Mai J, Higle JL (1994b) Solution of large scale stochastic programs with stochastic decomposition algorithms. Hager WW, Hearn DW, Pardalos PM, eds. *Large Scale Optimization: State of the Art* (Springer, Boston), 388–410.

Shapiro A, Homem-de Mello T (1998) A simulation-based approach to two-stage stochastic programming with recourse. *Math. Programming* 81(3):301–325.

Tsay R (2005) *Analysis of Financial Time Series*, Wiley Series in Probability and Statistics, 2nd ed. (Wiley-Interscience, Hoboken, NJ).

USC3DLAB (2019) Two-stage stochastic decomposition. Accessed July 21, 2019, https://github.com/USC3DLAB/SD.git.

Van Slyke RM, Wets RJB (1969) L-shaped linear programs with applications to optimal control and stochastic programming. *SIAM J. Appl. Math.* 17(4):638–663.

You F, Wassick JM, Grossmann IE (2009) Risk management for a global supply chain planning under uncertainty: Models and algorithms. *AIChE J.* 55(4):931–946.

Copyright 2021, by INFORMS, all rights reserved. Copyright of Journal on Computing is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.